

Complete Linux Mastery Course for DevOps Engineering

From Absolute Beginner to Production-Level Linux for DevOps/SRE (Ubuntu 24.04 LTS)

Prepared for Aryan

August 2026

Contents

1	How to Use This Course	6
2	PART 1 — Linux Fundamentals	7
2.1	1.1 What is an Operating System?	7
2.2	1.2 What is Linux? Linux vs UNIX	7
2.3	1.3 The Linux Kernel and GNU/Linux	7
2.4	1.4 Linux Distributions	7
2.5	1.5 Kernel Space vs User Space	8
2.6	1.6 System Calls	8
2.7	1.7 Shell, Terminal, CLI vs GUI, Bash	9
2.7.1	The full request path	9
3	PART 2 — Linux Installation & Environments	10
3.1	2.1 Ways to Run Linux	10
3.2	2.2 The Linux Boot Sequence	10
4	PART 3 — Linux Filesystem	12
4.1	3.1 The Filesystem Hierarchy Standard (FHS)	12
4.1.1	Directory-by-directory, with DevOps relevance	12
4.2	3.2 Paths	13
4.3	3.3 Hidden Files and Extensions	13
4.4	3.4 Inodes, Hard Links, Symbolic Links	13
4.5	3.5 File Metadata and Mount Points	14
4.6	4.1 Navigation	14
4.7	4.2 File Operations	15
4.8	4.3 Viewing Files	15
4.9	4.4 Text Processing	16
4.10	4.5 Pipelines and Redirection	17
5	PART 5 — File Permissions	18
5.1	5.1 The Permission Model	18
5.2	5.2 Changing Permissions and Ownership	18
5.2.1	Symbolic mode	18
5.2.2	Numeric (octal) mode	18
5.3	5.3 Special Permissions	19
5.4	5.4 ACLs (Access Control Lists)	20

6	PART 6 — Users & Groups	21
6.1	6.1 Core Concepts	21
6.2	6.2 Key System Files	21
6.3	6.3 User & Group Management Commands	21
6.4	6.4 How Linux Authentication Actually Works (Practical Level)	22
6.5	7.1 What Is a Process?	23
6.6	7.2 Foreground, Background, Daemons	23
6.7	7.3 Zombie and Orphan Processes	23
6.8	7.4 Process States	23
6.9	7.5 Process Commands	24
	6.9.1 ps aux vs ps -ef	24
6.10	7.6 Signals	24
6.11	7.7 Troubleshooting a High-CPU Process	25
7	PART 8 — systemd & Services	26
7.1	8.1 What Is systemd?	26
7.2	8.2 systemctl — Managing Services	26
7.3	8.3 Creating a Custom systemd Service	26
7.4	8.4 Production Troubleshooting with systemd	27
8	PART 9 — Logging	28
8.1	9.1 Where Logs Live	28
8.2	9.2 Logging Commands	28
8.3	9.3 Log Rotation	28
8.4	9.4 Investigating Common Failures via Logs	28
8.5	10.1 Storage Fundamentals	29
8.6	10.2 Mounting	29
	8.6.1 /etc/fstab — persistent mounts	30
8.7	10.3 Storage Commands	30
8.8	10.4 Filesystems: ext4, XFS, btrfs	31
8.9	10.5 Swap	31
9	PART 11 — Disk Troubleshooting	32
9.1	11.1 The Classic “Disk Is Full” Investigation	32
9.2	11.2 Inode Exhaustion (a Sneaky Variant)	32
9.3	11.3 “Deleted” Files Still Consuming Space	32
9.4	11.4 Log Files Consuming All Storage	32
10	PART 12 — Memory Management	33
10.1	12.1 Concepts	33
10.2	12.2 Commands	33
10.3	12.3 Investigating High Memory Usage / OOM Kills	33
11	PART 13 — CPU Management	35
11.1	13.1 Concepts	35
11.2	13.2 Understanding Load Average Properly	35
11.3	13.3 Commands	35
11.4	14.1 OSI Model and TCP/IP Model	36
11.5	14.2 Addressing	36
11.6	14.3 DNS and DHCP	36
11.7	14.4 ARP	37
11.8	14.5 TCP vs UDP, Ports, Sockets	37
11.9	14.6 HTTP, HTTPS, TLS, SSH	37
11.10	14.7 NAT, Proxies, Load Balancers	37
11.11	14.8 Networking Commands	37

11.124.9 Practical Troubleshooting: The Full Chain	38
12PART 15 — SSH	40
12.115.1 Core Commands	40
12.215.2 Public/Private Key Authentication	40
12.315.3 Configuration Files	40
12.415.4 SSH Hardening Checklist	40
12.515.5 Port Forwarding	41
12.615.6 Connecting to AWS EC2 Securely	41
13PART 16 — Firewalls	42
13.116.1 iptables, nftables, and ufw	42
13.216.2 Practical UFW Examples	42
13.316.3 Linux Firewall vs AWS Security Groups/NACLs	42
14PART 17 — Package Management	44
14.117.1 APT (Debian/Ubuntu)	44
14.217.2 YUM / DNF / RPM (RHEL, CentOS, Rocky, AlmaLinux, Amazon Linux)	44
14.318.1 Core Variables	45
14.418.2 Commands	45
14.518.3 Where Variables Get Set: .bashrc, .profile, /etc/environment	46
14.618.4 PATH Resolution and Command Lookup	46
15PART 19 — Bash Shell Scripting	47
15.119.1 Variables, Strings, Numbers, Arrays	47
15.219.2 Conditionals	47
15.319.3 Loops	47
15.419.4 Functions, Arguments, Exit Codes	48
15.519.5 Command Substitution, Arithmetic, Tests	48
15.619.6 Traps and Signals in Scripts	48
15.719.7 A Real DevOps Script: Health-Check + Restart	49
16PART 20 — Cron & Automation	50
16.120.1 cron and crontab	50
16.1.1Crontab syntax	50
16.220.2 at — One-Time Scheduled Jobs	50
16.320.3 systemd Timers (Modern Alternative to cron)	50
16.420.4 Practical Automation Examples	51
16.521.1 Principle of Least Privilege	51
16.621.2 Core Building Blocks (Recap)	51
16.721.3 SELinux vs AppArmor	52
16.821.4 Secrets and Password Policy	52
16.921.5 Security Updates and Audit Logs	52
16.101.6 Fail2ban	53
17PART 22 — Linux Performance	54
17.122.1 The Systematic Troubleshooting Framework	54
17.222.2 Tools	54
17.322.3 Diagnosing Each Bottleneck Type	54
18PART 23 — Advanced Linux Tools	56
18.124.1 What the Kernel Actually Manages	57
18.224.2 Kernel Modules	57
18.324.3 sysctl — Kernel Runtime Tuning	57
18.424.4 uname and dmesg	58

19PART 25 — /proc and /sys	59
19.125.1 What These Are	59
19.225.2 Practical /proc Files	59
19.325.3 /sys	59
20PART 26 — Linux Containers: The Foundation of Docker & Kubernetes	60
20.126.1 Why This Matters	60
20.226.2 Namespaces	60
20.326.3 cgroups (Control Groups)	61
20.426.4 Capabilities	61
20.526.5 Overlay Filesystems	61
20.626.6 The Chain, Explicitly	61
21PART 27 — Linux on AWS (EC2)	62
21.127.1 EC2 Linux Basics	62
21.227.2 SSH and IAM	62
21.327.3 Instance Metadata Service (IMDS) and IMDSv2	62
21.427.4 EBS, Security Groups, cloud-init, and Logs	62
21.527.5 Troubleshooting a Production EC2 Instance	63
22PART 28 — Linux with Docker	64
22.128.1 The Docker Daemon and Container Processes	64
22.228.2 Volumes and Networking	64
22.328.3 Logs and Resource Limits	64
23PART 29 — Linux with Kubernetes	66
23.129.1 The Linux Foundations Kubernetes Depends On	66
23.229.2 What Actually Happens When a Pod Runs on a Node	66
23.329.3 Signals in Kubernetes (tying back to Part 7.6)	66
23.430.1 Server Is Slow	67
23.530.2 CPU Is 100%	67
23.630.3 Memory Is Full	67
23.730.4 Disk Is Full	67
23.830.5 SSH Isn't Working	68
23.930.6 Website Isn't Accessible	68
23.100.7 Port Isn't Listening	68
23.110.8 DNS Isn't Resolving	68
23.120.9 Service Won't Start	68
23.130.10 Service Keeps Restarting (Crash Loop)	68
23.140.11 Nginx Returns 502 (Bad Gateway)	69
23.150.12 Nginx Returns 504 (Gateway Timeout)	69
23.160.13 Permission Denied	69
23.170.14 Process Is Stuck	69
23.180.15 Zombie Process	70
23.190.16 High Load Average	70
23.200.17 Network Latency	70
23.210.18 Packet Loss	70
23.220.19 Container Cannot Connect to Another Container	70
23.230.20 Application Crashes	70
23.240.21 Log Files Fill the Disk	71
23.250.22 EC2 Instance Cannot Be Accessed	71
23.260.23 Docker Container Keeps Restarting	71
23.27Navigation	71
23.28Files	71
23.29Text Processing	71

23.30	Permissions	72
23.31	Users	72
23.32	Processes	72
23.33	Services (systemd)	72
23.34	Networking	72
23.35	Storage	73
23.36	Monitoring	73
23.37	Packages	73
23.38	SSH	73
23.39	Security	73
23.40	Archives & Compression	73
23.41	Scheduling	74
23.42	Debugging	74
23.43	System Information	74
23.44	Linux Fundamentals	74
23.45	Filesystem	74
23.46	Permissions	75
23.47	Processes	75
23.48	Networking	75
23.49	SSH	75
23.50	systemd	76
23.51	Storage	76
23.52	Memory	76
23.53	CPU	76
23.54	Security	76
23.55	Bash	76
23.56	Troubleshooting	77
23.57	Docker/Linux	77
23.58	Kubernetes/Linux	77
23.59	AWS/Linux	77
23.60	Foundations (Labs 1-10)	77
23.61	Processes & Services (Labs 11-20)	78
23.62	Automation & Troubleshooting (Labs 21-30)	78
23.63	Storage & Filesystem Depth (Labs 31-35)	79
23.64	Networking & Security Depth (Labs 36-42)	79
23.65	Containers & Cloud (Labs 43-50+)	79
23.66	Project Goal	80
23.67	Architecture	80
23.68	Step-by-Step	80
24	Complete Linux Roadmap	82
25	30-Day Linux Learning Plan	82
25.1	Daily Practice Schedule (suggested, ~1.5-2 hrs/day)	83
26	Troubleshooting Checklist (Print/Keep Handy)	83

1 How to Use This Course

This is a complete, from-zero-to-production Linux reference built specifically for **DevOps/SRE work** — not for a certification exam. It is organized into 34 parts that build on each other:

- **Parts 1-13:** Core Linux internals — how the OS, filesystem, processes, permissions, users, storage, memory, and CPU actually work.
- **Parts 14-20:** Operating Linux from the command line — networking, SSH, firewalls, packages, environment variables, Bash scripting, and automation.
- **Parts 21-23:** Securing and tuning Linux systems.
- **Parts 24-29:** The Linux internals that Docker, Kubernetes, and AWS EC2 are built on.
- **Part 30:** Realistic production troubleshooting scenarios.
- **Parts 31-34:** Command cheat sheet, interview prep, 50+ hands-on labs, and a full capstone DevOps project.
- **Back matter:** Linux roadmap, 30-day learning plan, and troubleshooting checklist.

Notation used throughout:

- Commands are shown in monospace.
- **[DANGEROUS]** marks any command that can delete, overwrite, or corrupt data — read the warning before running it.
- “DevOps use case” boxes connect the raw Linux concept to something you will actually do as a DevOps/Cloud engineer (EC2, Docker, Kubernetes, CI/CD, monitoring).
- Every major concept ends with **common mistakes** and, where relevant, a **troubleshooting** angle.

Use Ubuntu 24.04 LTS (a real VM, WSL2, or an EC2 instance) side-by-side with this document. Reading Linux is not learning Linux — type every command yourself.

2 PART 1 — Linux Fundamentals

2.1 1.1 What is an Operating System?

An Operating System (OS) is the software layer that sits between raw hardware (CPU, RAM, disk, network card) and the programs you run. It has three core jobs:

1. **Resource management** — decides which process gets CPU time, how much RAM each process can use, and in what order disk/network requests are served.
2. **Abstraction** — gives programs a simple, consistent way to talk to hardware (e.g., `write()` to a file, instead of manipulating disk sectors directly).
3. **Isolation and protection** — keeps processes from stepping on each other's memory, and keeps users from accessing files they don't own.

Windows, macOS, and Linux are all operating systems, but Linux is overwhelmingly what runs the servers, containers, and cloud infrastructure that DevOps engineers manage — the vast majority of public cloud VMs and virtually all Docker/Kubernetes nodes run Linux.

2.2 1.2 What is Linux? Linux vs UNIX

Linux is a **kernel** — a single program that manages hardware, memory, processes, and system calls. It was created by Linus Torvalds in 1991, inspired by UNIX design principles.

UNIX is a family of operating systems (and a trademark) developed at Bell Labs starting in 1969. Commercial UNIX variants include Solaris, AIX, and HP-UX. Linux is not derived from UNIX source code — it is a from-scratch, UNIX-like kernel that follows the same philosophy: “everything is a file,” small composable tools, and a hierarchical filesystem.

	UNIX	Linux
Origin	AT&T Bell Labs, 1969	Linus Torvalds, 1991
License	Proprietary (mostly)	Open source (GPL)
Kernel + OS	Bundled together historically	Kernel only — distros add everything else
Examples	Solaris, AIX, HP-UX, macOS (BSD-derived)	Ubuntu, RHEL, Debian, Amazon Linux

2.3 1.3 The Linux Kernel and GNU/Linux

The **kernel** alone cannot give you a usable system — it has no shell, no `ls`, no compiler. The **GNU Project** (started by Richard Stallman) supplied the userland tools: the shell (`bash`), core utilities (`ls`, `cp`, `grep`, `sed`), compilers (`gcc`), and libraries. When you combine the Linux kernel with GNU tools plus a package manager and installer, you get a **GNU/Linux distribution**.

Kernel responsibilities: - Process scheduling (deciding which process runs on which CPU core, when) - Memory management (virtual memory, paging, allocation) - Device drivers (talking to disks, NICs, GPUs) - Filesystem implementation (`ext4`, `XFS`, etc.) - Networking stack (TCP/IP implementation) - System call interface (the API programs use to ask the kernel to do things)

2.4 1.4 Linux Distributions

A **distribution** (“**distro**”) = Linux kernel + GNU tools + package manager + default configuration + (often) a desktop environment. Distros differ mainly in **package management**, **release philosophy**, and **default configuration** — the kernel and most core tools are shared.

Distro	Family	Pkg mgr	Release model	Common DevOps use
Ubuntu	Debian	APT	LTS every 2 yrs, 5 yr support	Most common dev/server distro; default for many Docker base images
Debian	Debian	APT	Stable, conservative	Base for Ubuntu; used where extreme stability matters
RHEL	RHEL	DNF/YUM	Fixed, paid support	Enterprise servers, regulated industries
Rocky / Alma	RHEL	DNF/YUM	Free 1:1 RHEL rebuilds	Free alternative to RHEL, same ecosystem
Amazon Linux	RHEL-like	DNF/YUM	AWS-maintained	Default choice for AWS EC2 workloads

Practical implication for you: commands are ~95% identical across distros. The differences that matter day-to-day are: package manager syntax (apt vs dnf/yum), default user (ubuntu vs ec2-user), and firewall tooling (ufw on Ubuntu vs firewallld on RHEL family).

2.5 1.5 Kernel Space vs User Space

Linux divides memory and privilege into two rings:

- **Kernel space:** where the kernel runs, with full access to hardware and memory. Bugs here can crash the whole machine.
- **User space:** where normal programs (bash, nginx, your Python app) run, with restricted access. A crash here only kills that process.

Programs in user space cannot directly touch hardware. Instead they ask the kernel to do it on their behalf via a **system call**.

2.6 1.6 System Calls

A **system call (syscall)** is the formal, controlled entry point from user space into the kernel. Examples: `open()`, `read()`, `write()`, `fork()`, `execve()`, `socket()`, `connect()`.

When you run `cat file.txt`: 1. The cat program (user space) calls `open("file.txt")`. 2. This triggers a syscall — the CPU switches from user mode to kernel mode. 3. The kernel's filesystem code checks permissions, finds the file on disk, and returns a file descriptor. 4. cat then calls `read()` repeatedly (more syscalls) to pull data, and `write()` to print it to your terminal.

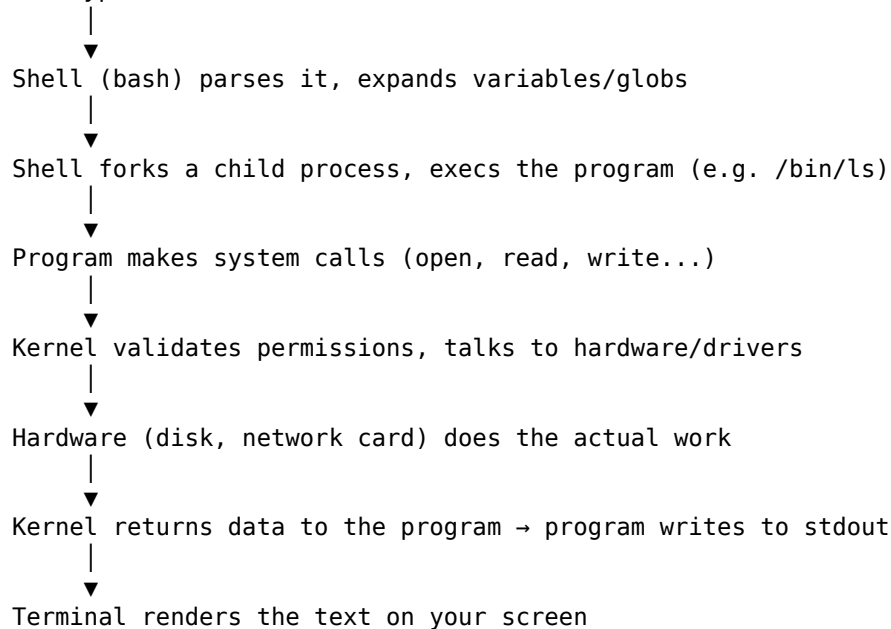
You can literally watch this happen with `strace cat file.txt` (covered in Part 23) — this is one of the most useful DevOps debugging skills there is.

2.7 1.7 Shell, Terminal, CLI vs GUI, Bash

- **Terminal (terminal emulator):** the window/program that displays text input/output (e.g., GNOME Terminal, iTerm2, Windows Terminal, or an SSH session).
- **Shell:** the program that reads the commands you type, interprets them, and asks the kernel to execute them. Examples: bash, zsh, sh, dash, fish.
- **Bash** (Bourne Again SHell) is the default shell on almost every Linux server and the one this course focuses on.
- **CLI vs GUI:** servers almost never run a GUI — every EC2 instance, every Docker container, every Kubernetes node is managed over CLI/SSH. This is why command-line fluency is non-negotiable for DevOps.

2.7.1 The full request path

You type a command



This chain — **User → Shell → System Call → Kernel → Hardware** — is the mental model behind almost every troubleshooting question in this course. When something is “slow” or “broken,” you are almost always diagnosing which layer of this chain is failing.

Common mistakes: confusing the terminal (the window) with the shell (the interpreter) with the kernel (the core OS) — they’re three different, stacked things.

3 PART 2 — Linux Installation & Environments

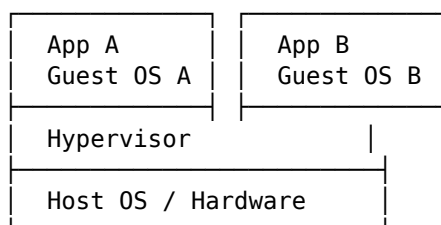
3.1 2.1 Ways to Run Linux

Method	What it is	When DevOps engineers use it
Bare-metal install	Linux installed directly on physical hardware	On-prem servers, dedicated hosts
Virtual Machine (VM)	Linux runs inside a hypervisor (VirtualBox, VMware, KVM)	Local dev/test, on-prem virtualization
WSL2	Real Linux kernel running inside Windows via a lightweight VM	Local Windows dev machines
Cloud Linux server	A VM provisioned by a cloud provider	AWS EC2, GCP Compute Engine, Azure VM
Docker container	A process isolated with namespaces/cgroups, sharing the host kernel	CI/CD, microservices, local app testing

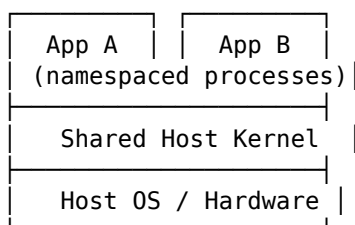
Containers vs VMs — the distinction that matters most for DevOps:

- A **VM** virtualizes hardware. Each VM has its *own full kernel*, boots independently, and is isolated by the hypervisor. Heavier, slower to start (seconds-minutes), stronger isolation.
- A **container** virtualizes the *operating system*, not the hardware. All containers on a host **share the host's kernel**, and are isolated using kernel features called **namespaces** and **cgroups** (Part 26). Lighter, starts in milliseconds, weaker isolation boundary than a VM.

VM:



Container:



3.2 2.2 The Linux Boot Sequence

Understanding boot order is essential for diagnosing “server won’t come up” incidents.

1. BIOS/UEFI → 2. Bootloader (GRUB) → 3. Kernel + initramfs
→ 4. systemd (PID 1) → 5. Targets/services start → 6. Login prompt
1. **BIOS/UEFI** — firmware built into the motherboard. Runs POST (Power-On Self-Test), initializes basic hardware, then hands control to the bootloader found on the boot device. UEFI is the modern replacement for legacy BIOS — faster, supports larger disks, and uses the GPT partition scheme.
2. **Bootloader (GRUB)** — GRand Unified Bootloader. Presents the boot menu (if you’ve ever seen “Ubuntu / Advanced options”), loads the selected **kernel image** and **initramfs** into memory, and passes control to the kernel.

3. **Kernel + initramfs** — the kernel starts running. `initramfs` (initial RAM filesystem) is a small temporary filesystem loaded into RAM containing just enough drivers/tools to mount the *real* root filesystem (which might be on LVM, RAID, or an encrypted disk that needs special drivers before it can be reached).
4. **systemd (PID 1)** — once the real root filesystem is mounted, the kernel starts `/sbin/init`, which on modern distros is **systemd**. This is always **PID 1** and is the ancestor of every other process on the system.
5. **Targets and services** — systemd works through dependency-ordered **targets** (roughly equivalent to old-style “runlevels”) — starting networking, mounting filesystems, launching services like `sshd`, `docker`, `nginx`, and your app.
6. **Login** — a `getty` process presents a login prompt on the console, or (for servers) `sshd` starts accepting SSH connections.

DevOps use case: when an EC2 instance fails a status check or won’t SSH, the boot sequence is your diagnostic checklist — is it a hardware/hypervisor issue (AWS console screenshot of console output shows GRUB or kernel panic), a filesystem issue (`fsck` errors during boot), or a systemd/service issue (boots fine but `sshd` or `cloud-init` failed)?

Common mistakes: editing `/etc/fstab` incorrectly (Part 10) is one of the most common ways to make a server fail to boot — a bad entry there can drop you into an emergency/rescue shell.

4 PART 3 — Linux Filesystem

4.1 3.1 The Filesystem Hierarchy Standard (FHS)

Linux has **one single directory tree** rooted at / — unlike Windows' C:\, D:\. Every disk, partition, network share, or USB drive gets **mounted** as a subdirectory somewhere in this one tree.

```
/
├── bin      -> usr/bin    (essential user command binaries; symlink on modern systems)
├── sbin     -> usr/sbin   (essential system admin binaries)
├── boot     (kernel, initramfs, GRUB config)
├── dev      (device files — /dev/sda, /dev/null, /dev/tty)
├── etc      (system-wide configuration files)
├── home     (personal directories for regular users, e.g. /home/aryan)
├── lib      -> usr/lib    (shared libraries needed by /bin and /sbin)
├── media    (auto-mounted removable media)
├── mnt      (conventional place to manually mount things temporarily)
├── opt      (optional/third-party software, self-contained packages)
├── proc     (virtual — live kernel/process info, not real files on disk)
├── root     (home directory for the root user — NOT the / root)
├── run      (runtime data since last boot — PID files, sockets)
├── srv      (data served by this system, e.g. web/ftp content)
├── sys      (virtual — kernel/device info, similar to /proc)
├── tmp      (world-writable temp storage, often cleared on reboot)
├── usr      (the bulk of installed software: usr/bin, usr/lib, usr/share)
├── var      (variable data: logs, caches, mail, databases, /var/log,
└── ↪ /var/www)
```

4.1.1 Directory-by-directory, with DevOps relevance

Dir	What lives there	DevOps relevance
/etc	Config files (/etc/passwd, /etc/nginx/nginx.conf, /etc/ssh/sshd_config, /etc/fstab)	You will edit files here constantly to configure services
/var/log	Application and system logs	First place to look during any incident
/home	User home directories, SSH keys (~/.ssh)	Where you'll keep dotfiles, scripts, .bashrc
/opt	Third-party software installed outside the package manager (e.g., some monitoring agents)	Common install location for CloudWatch agent, Datadog agent
/tmp	Scratch space	Watch it — it can fill up and cause disk-full incidents
/proc, /sys	Live kernel/process state (virtual, not on disk)	Source of truth for monitoring tools (Part 25)
/boot	Kernel images, GRUB config	Rarely touched, critical during boot troubleshooting
/srv, /var/www	Served application/website content	Where you'd point Nginx's root directive

Dir	What lives there	DevOps relevance
/dev	Device files (/dev/sda1 = a disk partition, /dev/null = discard)	Used in disk/storage commands (Part 10)

4.2 3.2 Paths

- **Absolute path:** starts from /, unambiguous regardless of your current location. E.g., /home/aryan/project/app.py.
- **Relative path:** starts from your **current working directory**. E.g., project/app.py (if you're already in /home/aryan).
- . = current directory, .. = parent directory, ~ = your home directory, - (with cd) = previous directory.

```
cd /var/log      # absolute
cd ../etc        # relative – go up one level, then into etc
cd ~             # your home directory
cd -             # jump back to previous directory
```

4.3 3.3 Hidden Files and Extensions

- Any file/directory starting with . is **hidden** from a normal ls (needs ls -a). Convention for config/dotfiles: .bashrc, .ssh/, .gitconfig.
- Linux **does not require file extensions** to determine file type (unlike Windows) — the kernel and most programs look at the file's actual content/**magic bytes**, not the name. .txt, .sh, .py are conventions for humans and editors, not requirements. This is why file somename (no extension) still correctly reports “ELF executable” or “ASCII text.”

4.4 3.4 Inodes, Hard Links, Symbolic Links

An **inode** is a data structure on the filesystem storing all metadata about a file — permissions, owner, size, timestamps, and pointers to the actual data blocks on disk — **everything except the filename**. The filename lives in the parent **directory entry**, which just maps a name to an inode number.

```
ls -li file.txt    # show the inode number
stat file.txt      # show full inode metadata
```

This “filename is separate from the file” design is exactly what makes links possible:

- **Hard link** (ln target linkname): a second directory entry pointing to the **same inode**. Both names are equally “real” — deleting one leaves the data intact under the other name. Cannot cross filesystems/partitions, cannot link directories.
- **Symbolic link / symlink** (ln -s target linkname): a small special file that just contains the **path string** to another file. Can cross filesystems, can point to directories, but breaks (“dangling symlink”) if the target is deleted or moved.

```
ln -s /var/log/nginx/access.log ~/nginx-access # symlink – common for quick shortcuts
ln /data/config.yml /data/config-backup.yml    # hard link – same inode, two names
```

DevOps use case: the /bin -> usr/bin structure you saw above is itself a symlink (modern “usrmerge” layout). Symlinks are also how tools like update-alternatives, Docker’s layered filesystem shortcuts, and versioned release directories (current -> releases/v1.4.2, a very common **zero-downtime deployment pattern**) work.

4.5 3.5 File Metadata and Mount Points

`stat file.txt` shows: size, inode number, permissions, owner/group, link count, and three timestamps: - **atime** — last access (read) - **mtime** — last modification (content changed) - **ctime** — last metadata/inode change (permissions, owner — NOT “creation time,” a common misconception)

A **mount point** is a directory that becomes the entry point for another filesystem (a separate disk partition, a network share, or a virtual filesystem like `/proc`). `df -h` shows what’s mounted where and how full each mount is — one of the most-used DevOps commands, covered fully in Part 10.

Common mistakes: - Assuming file extensions determine behavior — Linux doesn’t care if you name a script `deploy.txt`; if it’s executable and has a valid shebang, it runs. - Confusing hard links with symlinks — hard links share an inode (can’t tell them apart without `ls -li`), symlinks are visibly separate files (`ls -l` shows `-> target`). - Deleting a file that a running process still has open doesn’t free disk space immediately — the kernel keeps the inode alive until the last open file descriptor closes (classic “disk full but `du` doesn’t show why” scenario — covered in Part 11). # PART 4 — The Complete Linux Command Line

4.6 4.1 Navigation

Command	Purpose	Key options	Example	DevOps use case
<code>pwd</code>	Print working directory		<code>pwd</code>	Confirm where a deploy script actually ran from
<code>ls</code>	List directory contents	<code>-l</code> long, <code>-a</code> all incl. hidden, <code>-h</code> human sizes, <code>-t</code> sort by time, <code>-R</code> recursive	<code>ls -lath /var/log</code>	Check log directory sizes/recency during an incident
<code>cd</code>	Change directory	<code>-</code> previous dir, <code>~</code> home	<code>cd /etc/nginx</code>	Navigate to config directories
<code>tree</code>	Visual directory tree	<code>-L n</code> depth limit	<code>tree -L 2 /opt/app</code>	Understand an unfamiliar app’s layout quickly
<code>find</code>	Search files by criteria	<code>-name</code> , <code>-type</code> , <code>-mtime</code> , <code>-size</code> , <code>-exec</code>	<code>find /var/log -name "*.log" -mtime +7</code>	Find/clean logs older than N days
<code>locate</code>	Fast filename search via prebuilt index	<code>updatedb</code> refreshes index	<code>locate nginx.conf</code>	Quick lookup (index may be stale — <code>find</code> is more reliable on servers)

find deep dive (the single most powerful navigation command for DevOps):

```
find /var/log -type f -name "*.log" -mtime +30 -size +50M # files >50MB, older than
↳ 30 days
find / -xdev -type f -perm -4000 # find all SUID binaries
↳ (security audit)
```

```
find /app -type f -name "*.py" -exec grep -l "TODO" {} \; # find files containing
↳ "TODO"
find . -type d -empty -delete # [!] DANGEROUS – deletes every empty directory found
```

-exec runs a command on every match; {} is replaced by the filename, \; terminates the command per-file (use + instead of \; to batch files into fewer invocations — much faster on large result sets).

Common mistakes: find with -delete or -exec rm and no -name/-type filter tested first — always run the find **without** the destructive action first, check the output, *then* add -delete.

4.7 4.2 File Operations

Command	Purpose	Key options	Example
touch	Create empty file / update timestamp		touch app.log
mkdir	Create directory	-p create parents as needed	mkdir -p /opt/app/releases/v1
cp	Copy files/dirs	-r recursive, -p preserve attrs, -v verbose	cp -rp /etc/nginx /backup/nginx- \$(date +%F)
mv	Move/rename		mv app.log app.log.bak
rm	Remove files/dirs	-r recursive, -f force, -i confirm	[!] rm -rf /opt/old-release
rmdir	Remove empty directory only		rmdir emptydir
ln file	Create links (Part 3.4) Identify file type by content	-s symbolic	ln -s current app file
stat	Show detailed file metadata		stat /etc/passwd

[DANGEROUS]: rm -rf permanently deletes without a trash bin. rm -rf / is blocked by modern rm unless you pass --no-preserve-root, but rm -rf /* in the wrong directory, or rm -rf \$VAR/ where \$VAR is unexpectedly empty (expanding to rm -rf /), are real production disasters. **Always** run ls on the target first, and consider -i for interactive confirmation on risky deletes.

4.8 4.3 Viewing Files

Command	Purpose	Example
cat	Print whole file to stdout	cat /etc/os-release
tac	Like cat but reversed (last line first)	tac access.log
less	Paginated viewer, searchable, doesn't load whole file into memory	less /var/log/syslog
more	Older, simpler pager (mostly superseded by less)	more file.txt

Command	Purpose	Example
head	First N lines	head -n 50 app.log
tail	Last N lines; -f follows live updates	tail -f /var/log/nginx/access.log
nl	Print with line numbers	nl script.sh
strings	Extract printable text from a binary file	strings /usr/bin/mystery_binary grep -i version

tail -f is one of the most-used DevOps commands — live-tailing a log while reproducing an issue. less +F filename gives similar behavior with less's search capability.

Common mistakes: using cat on a multi-GB log file instead of less/tail — floods your terminal and can be slow; cat also loads output all at once rather than paginating.

4.9 4.4 Text Processing

Command	Purpose	Example
grep egrep	Search text by pattern grep -E, extended regex	grep -ri "error" app.log egrep "error fail exception" app.log
sed	Stream editor — find/replace, transform text	sed -i 's/DEBUG/INFO/g' config.yml
awk	Field-based text processing / mini programming language	awk '{print \$1, \$7}' access.log
cut	Extract columns/fields	cut -d: -f1 /etc/passwd
sort	Sort lines	sort -k2 -n data.txt
uniq	Remove/count adjacent duplicate lines (needs sorted input)	sort ips.txt uniq -c sort -rn
tr	Translate/delete characters	cat file tr 'a-z' 'A-Z'
wc xargs	Count lines/words/bytes Build/execute commands from stdin	wc -l access.log find . -name "*.tmp" xargs rm
tee	Write stdout to a file and pass it through	command tee output.log
column	Format text into aligned columns	column -t -s, data.csv

Key options worth memorizing: - grep -r recursive, -i ignore case, -n line numbers, -v invert match, -c count matches, -E extended regex, -A/-B/-C n lines of context after/before/around. - sed -i edits the file **in place** — [!] no undo, always test without -i first or keep a .bak (sed -i.bak ...). - awk '{print \$N}' — \$0 is the whole line, \$1..\$N are whitespace-delimited fields, -F, changes the delimiter (for CSVs).

Real DevOps one-liner examples:

```
# Top 10 IPs hitting your server (classic log analysis)
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
```



```
# Find all 5xx errors in an Nginx log
grep -E ' 5[0-9]{2} ' access.log | tail -50

# Replace an environment placeholder across all config files
grep -rl "{{ENV}}" /app/config | xargs sed -i 's/{{ENV}}/production/g'

# Count how many times each error type appears
grep -oP 'ERROR: \K.*' app.log | sort | uniq -c | sort -rn
```

4.10 4.5 Pipelines and Redirection

A **pipeline** connects the stdout of one command to the stdin of the next:

```
command1 | command2 | command3
```

Each stage runs concurrently, streaming data — this is why `tail -f app.log | grep ERROR` shows matching lines live, rather than waiting for the file to end.

Every process has three standard streams: - **stdin (0)** — input, defaults to your keyboard/terminal. - **stdout (1)** — normal output. - **stderr (2)** — error output — kept *separate* from stdout so errors don't get mixed into piped data.

Operator	Meaning
>	Redirect stdout to a file, overwriting it
>>	Redirect stdout to a file, appending
<	Redirect a file as stdin
2>	Redirect stderr to a file, overwriting
2>>	Redirect stderr, appending
&> or >file 2>&1	Redirect both stdout and stderr to the same file
&	Run the preceding command in the background
&&	Run next command only if previous succeeded (exit code 0)
\ \	Run next command only if previous failed
;	Run next command regardless of previous result
\	Pipe stdout of one command into stdin of next

```
./build.sh > build.log 2>&1 & # run in background, capture all output
./deploy.sh && echo "Deploy OK" || echo "Deploy FAILED"
apt update; apt upgrade -y # run sequentially regardless of update's result
```

Common mistakes: `command > file 2>&1` vs `command 2>&1 > file` behave **differently** — order matters! The first redirects stdout to the file, then points stderr at “wherever stdout currently points” (the file) → both go to the file, correct. The second points stderr at the *current* stdout (the terminal) first, *then* redirects stdout to the file → stderr still prints to terminal, stdout goes to the file. Always put `2>&1` **after** the stdout redirect.


```

chmod 700 private_key    # rwx----- — owner only (typical for SSH private keys,
↳ secrets)
chmod 600 id_rsa         # rw----- — owner read/write only, no execute (SSH keys,
↳ credential files)

```

DevOps use case: SSH will *refuse* to use a private key if its permissions are too open (chmod 600 ~/.ssh/id_rsa is one of the most common “why won’t SSH work” fixes). Deployment scripts need +x to run; config files with secrets should be 600, never 644 or 777.

5.3 5.3 Special Permissions

Bit	Symbolic	Numeric prefix	Effect
SUID	chmod u+s file	4 (e.g. 4755)	When executed, the process runs with the file owner’s privileges, not the caller’s (e.g., /usr/bin/passwd is SUID root so any user can update /etc/shadow through it)
SGID	chmod g+s file	2 (e.g. 2755)	On a file: runs with the group’s privileges. On a directory : new files created inside inherit the directory’s group (very useful for shared team directories)
Sticky bit	chmod +t dir	1 (e.g. 1777)	On a directory: only the file’s owner (or root) can delete/rename it, even if others have write access — this is exactly how /tmp (mode 1777) stays safe as a shared world-writable directory

```

chmod 2775 /srv/shared-team-dir    # SGID: everyone in the group gets consistent group
↳ ownership on new files
ls -ld /tmp                        # drwxrwxrwt — note the trailing 't' = sticky bit

```

Security note: SUID binaries are a classic privilege-escalation target — find / -perm -4000 -type f 2>/dev/null is a standard security audit command to list every SUID binary on a system.

5.4 5.4 ACLs (Access Control Lists)

Standard permissions only support **one** owner and **one** group. When you need finer-grained permissions (e.g., “user X gets read-write, but no one else in the group does”), use ACLs:

```
getfacl file.txt
setfacl -m u:deploy_bot:rw file.txt      # grant a specific user rw, beyond the normal
↪ owner/group/other model
setfacl -x u:deploy_bot file.txt         # remove that specific grant
```

Common mistakes: - `chmod 777` “to make it work” — this is a security anti-pattern; it grants full read/write/execute to *everyone*, including any other process/user on a shared host or any process in a container that mounts the volume. Diagnose the *actual* required permission instead. - Forgetting that directory `x` is required to even `cd` into it or reach files by name inside it — a directory with `r--` but no `x` lets you list filenames but not access/enter them. - Editing a file as root via `sudo` and leaving it owned by root, then wondering why the application (running as a non-root service user) can’t read its own config.

6 PART 6 — Users & Groups

6.1 6.1 Core Concepts

- **User**: an identity with a unique numeric **UID**. root is UID 0 (unrestricted). Regular users typically start at UID 1000.
- **Group**: a collection of users sharing a **GID**, used to share permissions (Part 5).
- Every process runs as some user/group — this is the basis of the entire Linux security model.

6.2 6.2 Key System Files

File	Contains	Notes
/etc/passwd	Username, UID, GID, home dir, default shell (one line per user)	World-readable; no passwords stored here despite the name (historical)
/etc/shadow	Actual hashed passwords, expiry policy	Readable only by root — the sensitive file
/etc/group	Group names, GIDs, member lists	World-readable
/etc/sudoers	Who can run sudo, and what they're allowed to run as	Edit only with visudo — it syntax-checks before saving, preventing you from locking yourself out

```
cat /etc/passwd | grep aryan
# aryan:x:1001:1001:Aryan:/home/aryan:/bin/bash
# |      |      |      |      |      |
# |      |      |      |      |      | login shell
# |      |      |      |      |      | home directory
# |      |      |      |      |      | comment/full name (GECOS field)
# |      |      |      |      |      | primary GID
# |      |      |      |      |      | UID
# |      |      |      |      |      | password placeholder ('x' = actual hash is in /etc/shadow)
```

6.3 6.3 User & Group Management Commands

Command	Purpose	Example
useradd	Create a user (low-level, minimal defaults)	sudo useradd -m -s /bin/bash deploy
adduser	Friendlier, interactive wrapper (Debian/Ubuntu)	sudo adduser deploy
usermod	Modify an existing user	sudo usermod -aG docker aryan
userdel	Delete a user	sudo userdel -r deploy (-r also removes home dir)
passwd	Set/change a password	sudo passwd deploy
groupadd	Create a group	sudo groupadd devops
groupmod	Modify a group	sudo groupmod -n newname oldname

Command	Purpose	Example
groupdel groups	Delete a group Show groups a user belongs to	sudo groupdel devops groups arian
id	Show UID/GID and all group memberships	id arian
who whoami	Who is currently logged in Print current effective username	who whoami
w	Logged-in users + what they're running	w
last lastlog	Login history Last login time per user	last -n 10 lastlog

Very common real pattern: adding yourself to the docker group so you don't need sudo for every Docker command:

```
sudo usermod -aG docker $USER      # -a = append (don't use without -a, or you'll
↪ REPLACE all other groups!)
newgrp docker                      # or log out/in for the group change to take effect
```

[!] Forgetting `-a` with `usermod -G` **replaces** all of a user's supplementary group memberships with just the one you specified — a classic mistake that silently removes access the user had.

6.4 6.4 How Linux Authentication Actually Works (Practical Level)

1. You attempt to log in (console, SSH, su, sudo).
2. PAM (Pluggable Authentication Modules) — the framework behind authentication — checks `/etc/shadow` for your hashed password (or, for SSH, your public key against `~/.ssh/authorized_keys`, see Part 15).
3. If the credential matches, the kernel creates a process running as your UID/GID, with your groups attached.
4. From that point on, **every permission check** (file access, sudo eligibility, resource limits) is based on that UID/GID, not your “identity” in any abstract sense — Linux security is fundamentally UID-based.
5. sudo checks `/etc/sudoers` (or files in `/etc/sudoers.d/`) to decide whether your user/group is allowed to run the requested command as the target user (usually root), then temporarily elevates just that one command.

DevOps use case: in production, you almost never SSH in as root. Standard practice: log in as a named user (or via IAM-mapped SSH certs), then use sudo for specific privileged actions — this preserves an audit trail (`journalctl/auth.log` records *which user* ran *which* sudo command).

Common mistakes: - Editing `/etc/sudoers` directly with a text editor instead of `visudo` — a syntax error can lock out all sudo access system-wide. - Deleting a user with `userdel` without `-r`, leaving orphaned home directories and files owned by a now-nonexistent UID (`ls -l` will show a raw number instead of a username). - Sharing one root/admin login across a whole team instead of individual users with sudo — destroys accountability and audit trails. # PART 7 — Process Management

6.5 7.1 What Is a Process?

A **process** is a running instance of a program — its own memory space, open file descriptors, and execution state. Every process has:

- **PID** — Process ID, a unique number.
- **PPID** — Parent Process ID — the process that created it (every process except PID 1 has a parent).
- Every process is created by an existing process calling `fork()` (duplicate itself) and then usually `execve()` (replace its memory with a new program) — this **fork-exec** model is how every process on Linux, including your shell launching `ls`, comes into existence.

```
systemd (PID 1)
├─ sshd
│   └─ bash (your login shell)
│       └─ vim
```

6.6 7.2 Foreground, Background, Daemons

- **Foreground process**: occupies your terminal; you can't type another command until it finishes (or you background/suspend it).
- **Background process**: runs without blocking your terminal — start with `&`, or suspend a foreground job with `Ctrl+Z` then `bg`.
- **Daemon**: a background process, typically started at boot, with no controlling terminal, usually named ending in `d` (`sshd`, `nginx` (via workers), `dockerd`). Daemons are what `systemd` manages as services (Part 8).

6.7 7.3 Zombie and Orphan Processes

- **Orphan process**: its parent exited before it did. The kernel automatically **re-parents** it to `systemd`/PID 1, which “reaps” it normally. Not usually a problem.
- **Zombie process**: has *finished executing* but still has an entry in the process table because its parent hasn't yet called `wait()` to collect its exit status. Shows as `<defunct>` in `ps`. A zombie itself uses almost no resources — but if a *buggy parent* never reaps its children, zombies accumulate and can exhaust the process table (PID limit), which **does** become a real production issue.

```
ps aux | grep defunct      # find zombies
# Fix: the PARENT process must be fixed to properly wait() on children,
# or restarted/killed — you cannot directly "kill" a zombie, it's already dead.
```

6.8 7.4 Process States

State	Meaning
R (Running)	Currently executing or ready to run on a CPU
S (Sleeping)	Waiting for an event (I/O, timer) — most idle processes are here
D (Uninterruptible sleep)	Waiting on I/O (usually disk) — often a sign of disk trouble if many processes stack up in this state
T (Stopped)	Suspended (e.g., via <code>Ctrl+Z</code> or <code>SIGSTOP</code>)
Z (Zombie)	Finished but not yet reaped by parent

6.9 7.5 Process Commands

Command	Purpose	Example
ps	Snapshot of processes	ps aux (BSD style, all processes) / ps -ef (UNIX style)
top	Live, refreshing process view	top
htop	Nicer interactive version of top (may need install)	htop
pstree	Show processes as a parent/child tree	pstree -p
pgrep	Find PIDs by name	pgrep nginx
pidof	Find PID(s) of a running program	pidof sshd
kill	Send a signal to a PID	kill -9 1234
killall	Send a signal to all processes matching a name	killall nginx
pkill	Like killall but pattern-based	pkill -f "python app.py"
jobs	List background/suspended jobs of current shell	jobs
bg	Resume a suspended job in the background	bg %1
fg	Bring a background job to foreground	fg %1
nohup	Run a command immune to hangup (survives terminal close)	nohup ./server.sh &
nice	Start a process with a given priority	nice -n 10 ./batch_job.sh
renice	Change priority of a running process	renice -n 5 -p 1234
wait	Block until background job(s) finish	wait

6.9.1 ps aux vs ps -ef

Both list all processes system-wide — different formatting conventions (BSD vs UNIX style), same underlying info. `ps aux` shows %CPU/%MEM directly; `ps -ef` shows PPID directly. Most engineers just pick one and memorize it — `ps aux` is more common.

```
ps aux --sort=-%cpu | head -10    # top 10 CPU-consuming processes
ps aux --sort=-%mem | head -10    # top 10 memory-consuming processes
```

6.10 7.6 Signals

A **signal** is an asynchronous notification sent to a process — the fundamental mechanism for stopping, interrupting, or configuring a running process without direct access to its memory.

Signal	Number	Meaning	Can be caught/ignored?
SIGTERM	15	“Please terminate gracefully” — the default and polite kill signal	Yes — well-behaved apps catch it to flush data, close connections
SIGKILL	9	Force-kill immediately, no cleanup	No — kernel kills it unconditionally
SIGINT	2	Interrupt (what Ctrl+C sends)	Yes
SIGHUP	1	Originally “terminal hung up”; commonly repurposed by daemons to mean “reload your config”	Yes
SIGSTOP	19	Pause the process	No (cannot be caught)
SIGCONT	18	Resume a stopped process	Yes

```
kill 1234          # sends SIGTERM by default – graceful
kill -9 1234       # [!] SIGKILL – last resort, app gets zero chance to clean up (can
↳ corrupt in-flight writes)
kill -HUP 1234     # common way to tell a daemon to reload config without restarting
```

DevOps use case: Kubernetes sends SIGTERM to a container’s main process on pod termination, waits up to `terminationGracePeriodSeconds`, then sends SIGKILL if it hasn’t exited — your application **must** handle SIGTERM to shut down cleanly (finish in-flight requests, close DB connections) or you’ll see connection errors during every rolling deploy.

6.11 7.7 Troubleshooting a High-CPU Process

```
top                # spot the PID with high %CPU
ps -p <PID> -o pid,ppid,cmd,%cpu,%mem # confirm what it actually is
strace -p <PID> -c  # (Part 23) summarize which syscalls it's spending time
↳ in
cat /proc/<PID>/status # detailed state
kill -TERM <PID>      # try graceful first
kill -9 <PID>         # last resort
```

Common mistakes: reaching for `kill -9` first instead of SIGTERM — this skips graceful shutdown hooks and can leave temp files, locks, or half-written data behind. Also: killing a process by name with `killall/pkill` on a shared host without verifying you’re matching the *right* process — pattern matches can be broader than intended.

7 PART 8 — systemd & Services

7.1 8.1 What Is systemd?

systemd is the init system and service manager on virtually all modern distros (Ubuntu, Debian, RHEL/CentOS/Rocky, Amazon Linux 2/2023). It is **PID 1** — the first userspace process, parent of everything, and responsible for starting/stopping/supervising every other service.

Key concepts: - **Unit**: the basic configuration object systemd manages — most commonly a **service** unit (.service), but also .timer (cron-like scheduling), .mount, .socket, .target (a grouping of units, like an old “runlevel”). - **Target**: a synchronization point / group of units (e.g., multi-user.target = normal multi-user boot without GUI, graphical.target = with GUI). - **Dependencies**: units declare what must start before/after them (Requires=, After=, Wants=).

7.2 8.2 systemctl — Managing Services

Command	Purpose
systemctl start <svc>	Start a service now (does not persist across reboot)
systemctl stop <svc>	Stop a service now
systemctl restart <svc>	Stop then start
systemctl reload <svc>	Ask the service to reload config without dropping connections (if supported — e.g., Nginx)
systemctl enable <svc>	Configure to start automatically at boot (does NOT start it now)
systemctl disable <svc>	Remove from auto-start at boot
systemctl status <svc>	Show current state, recent log lines, PID
systemctl daemon-reload	Re-read unit files after you’ve edited/added one
systemctl list-units --type=service	List all active service units
systemctl is-enabled <svc> / is-active <svc>	Quick boolean checks, scriptable

```
sudo systemctl enable --now nginx    # enable at boot AND start immediately (common
↳ combo)
systemctl status nginx
```

7.3 8.3 Creating a Custom systemd Service

A complete production example — running a Python app as a managed, auto-restarting, auto-starting service:

```
# /etc/systemd/system/myapp.service
[Unit]
Description=My Application
After=network.target

[Service]
Type=simple
User=appuser
```

```

Group=appuser
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure
RestartSec=5
StandardOutput=journal
StandardError=journal
Environment=ENV=production

[Install]
WantedBy=multi-user.target

```

```

sudo systemctl daemon-reload      # tell systemd about the new unit file
sudo systemctl enable --now myapp  # start now + on every future boot
sudo systemctl status myapp        # verify it's running
journalctl -u myapp -f             # watch its logs live

```

This gives you: **automatic startup on boot** (enable + [Install] section), **restart on failure** (Restart=on-failure), and **centralized logging** (StandardOutput/Error=journal, viewable via journalctl -u myapp) — the three things a production process manager must provide, all built into systemd with no extra tooling (no need for supervisord/pm2 on a plain Linux host).

7.4 8.4 Production Troubleshooting with systemd

```

systemctl status myapp             # is it running? what was the last exit code/reason?
journalctl -u myapp                # full log history for this unit
journalctl -u myapp -f             # follow live
journalctl -u myapp --since "10 min ago"
journalctl -u myapp -p err         # only error-priority-and-above lines
journalctl -u myapp -n 100 --no-pager # last 100 lines, non-interactive (good for
↳ scripts)

```

systemctl status output tells you a huge amount at a glance: Active: failed (Result: exit-code), the exact PID, memory usage, and the last few log lines — this single command is usually your **first** move when “a service is down.”

Common mistakes: - Editing a .service file and forgetting systemctl daemon-reload — systemd keeps using the old, cached unit definition until you reload. - Using Type=simple for a program that daemonizes/forks itself (should be Type=forking) — systemd loses track of the real PID and reports incorrect status. - Not setting Restart=on-failure — a crashing app just stays down until someone notices, instead of self-healing.

8 PART 9 — Logging

8.1 9.1 Where Logs Live

- **/var/log/** — the traditional home for system and application logs (/var/log/syslog or /var/log/messages, /var/log/auth.log, /var/log/nginx/, /var/log/dpkg.log).
- **journald** — systemd’s structured, binary logging system. Every service managed by systemd has its logs captured here automatically (no app code changes needed) and queried with `journalctl`.
- **syslog/rsyslog** — the older, still-widely-used **plain-text** logging protocol/daemon. Many systems run rsyslog *alongside* journald, forwarding journal entries into traditional /var/log/*.log files for compatibility with older tools.

8.2 9.2 Logging Commands

Command	Purpose
<code>journalctl</code>	Query the systemd journal
<code>dmesg</code>	Kernel ring buffer — boot messages, hardware/driver events, OOM killer activity
<code>tail -f</code>	Live-follow a plain-text log file
<code>grep</code>	Search log content
<code>less</code>	Page through large log files

```
journalctl -k                # kernel messages (same source as dmesg,
↪ journal-formatted)
journalctl --since "1 hour ago" --until "10 minutes ago"
journalctl -p err -b         # errors-and-above since last boot
journalctl --disk-usage      # how much space the journal itself is using
dmesg -T | tail -50          # human-readable timestamps, last 50 kernel lines
```

8.3 9.3 Log Rotation

Logs grow forever unless rotated. **logrotate** (config in /etc/logrotate.conf and /etc/logrotate.d/) handles this: compressing, archiving, and deleting old log files on a schedule, so /var/log doesn’t fill the disk.

```
# /etc/logrotate.d/myapp
/var/log/myapp/*.log {
    daily
    rotate 14
    compress
    missingok
    notifempty
    copytruncate
}
```

`copytruncate` is important for apps that don’t reopen their log file on their own — it copies the current log then truncates the original in place, so the app keeps writing to the same (now-empty) file descriptor.

8.4 9.4 Investigating Common Failures via Logs

Symptom	Where to look	Commands
Application crash	App's own log, journald if systemd-managed	<code>journalctl -u myapp -n 200</code> , check for stack traces/exit codes
SSH failure	<code>/var/log/auth.log</code> (Debian/Ubuntu) or <code>/var/log/secure</code> (RHEL family)	<code>sudo tail -f /var/log/auth.log</code> , look for "Failed password," "Connection closed"
Disk full	Any write-heavy service log directory	<code>df -h</code> , then <code>du -sh /var/log/*</code> to find the culprit
Service failure	journald	<code>systemctl status <svc></code> , <code>journalctl -u <svc> -p err</code>
Kernel problem	dmesg, journald kernel logs	<code>dmesg -T \</code> <code>tail</code> , look for OOM-killer lines, hardware errors, filesystem errors
Authentication failure	<code>/var/log/auth.log</code> or <code>/var/log/secure</code>	<code>grep "authentication failure" /var/log/auth.log</code>

Common mistakes: - Not rotating logs on a busy service → disk-full incidents that take down the whole host (other services can't write either, since they usually share the same root filesystem). - Looking only at the application's own log and missing that the *real* cause is upstream — e.g., an app "crash" that's actually the kernel OOM-killer terminating it (visible only in dmesg, not the app's log). - Forgetting `journalctl` output is paged by default in an interactive terminal but should use `--no-pager` in scripts/automation. # PART 10 — Storage & Disks

8.5 10.1 Storage Fundamentals

- **HDD** (Hard Disk Drive) — spinning magnetic platters, mechanically slower, cheap per GB.
- **SSD** (Solid State Drive) — flash memory, no moving parts, much faster random I/O.
- **NVMe** — a modern SSD interface connected directly over PCIe (not the older SATA bus) — very high throughput; most modern cloud instance storage (e.g., AWS gp3/io2, EC2 instance store) is NVMe-backed.
- **Block device** — any storage device accessed in fixed-size chunks ("blocks"), e.g., `/dev/sda` (whole disk), `/dev/sda1` (first partition), `/dev/nvme0n1`.
- **Partition** — a logical division of a physical/virtual disk, each of which can hold its own filesystem.
- **Filesystem** — the structure imposed *on top of* a partition/block device that actually organizes files/directories (ext4, XFS, btrfs).

8.6 10.2 Mounting

Mounting attaches a filesystem (on a partition, disk, or network share) to a directory (**mount point**) in the single Linux directory tree, making its contents accessible there.

```
sudo mount /dev/xvdf1 /mnt/data      # temporary — lost on reboot
sudo umount /mnt/data                # unmount (note: "umount", no 'n')
```

8.6.1 /etc/fstab — persistent mounts

/etc/fstab defines filesystems that should be mounted **automatically at boot**.

```
# <device/UUID>          <mount point>  <fs type>  <options>
↪ <dump> <pass>
UUID=1a2b3c4d-...        /data        ext4        defaults,noatime
↪ 0      2
/dev/xvdf1                /mnt/backup  xfs         defaults
↪ 0      2
```

- Use **UUID** rather than /dev/sdX device names where possible — device *names* can shift between boots (especially with multiple attached volumes), but the UUID never changes. Get it with blkid.
- dump (5th field) — legacy backup flag, almost always 0 now.
- pass (6th field) — fsck order at boot: 0 = never check, 1 = root filesystem (checked first), 2 = checked after root.

[DANGEROUS]: a bad line in /etc/fstab (wrong UUID, wrong filesystem type, typo) can prevent the system from booting into multi-user mode at all — it may drop to an emergency shell. **Always test a mount manually first**, and consider adding the nofail option (boots even if that mount fails) for non-critical volumes.

8.7 10.3 Storage Commands

Command	Purpose	Example
lsblk	List block devices as a tree	lsblk -f (shows filesystem type + UUID too)
blkid	Show UUIDs and filesystem types	sudo blkid
df	Disk space usage per mounted filesystem	df -h
du	Disk usage per file/directory	du -sh /var/log/*
mount	Mount a filesystem / list current mounts	mount \ grep xvdf
umount	Unmount	sudo umount /mnt/data
fdisk	Partition a disk (MBR/legacy-friendly, interactive)	[!] sudo fdisk /dev/xvdf
parted	Partition a disk (GPT-friendly, scriptable)	[!] sudo parted /dev/xvdf
mkfs	Create (format) a filesystem	[!] sudo mkfs.ext4 /dev/xvdf1
fsck	Check/repair a filesystem	sudo fsck /dev/xvdf1 (unmount first!)
swapon/swapoff	Enable/disable a swap device or file	sudo swapon /swapfile

[DANGEROUS]: mkfs.* **destroys all existing data** on the target partition — always double- and triple-check the device name (lsblk first!) before formatting; formatting the wrong disk is an unrecoverable, career-notable mistake. Similarly, fdisk/parted can wipe partition tables — never run them on a device you haven't confirmed with lsblk.

8.8 10.4 Filesystems: ext4, XFS, btrfs

FS	Characteristics	Common use
ext4	Mature, stable, default on Ubuntu/Debian	General-purpose default choice
XFS	Excellent for large files and high parallel I/O, default on RHEL/CentOS/Amazon Linux	Databases, large file workloads
btrfs	Copy-on-write, built-in snapshots and checksumming	Snapshot-heavy setups (some container/backup workflows)

8.9 10.5 Swap

Swap is disk space used as overflow when physical RAM is exhausted — the kernel moves inactive memory pages there. It prevents immediate OOM kills under memory pressure but is **much slower than RAM**; heavy swap usage is itself a performance red flag (see Part 12).

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab # persist across reboots
```

9 PART 11 — Disk Troubleshooting

9.1 11.1 The Classic “Disk Is Full” Investigation

```
df -h                                # Step 1: which mounted filesystem is actually full?
df -i                                # Step 2: is it actually FULL, or out of INODES? (see 11.2)
du -sh /* 2>/dev/null | sort -rh | head # Step 3: which top-level dir is the culprit?
du -sh /var/log/* 2>/dev/null | sort -rh | head # drill down further
find / -xdev -type f -size +500M 2>/dev/null # Step 4: find individual huge files
```

9.2 11.2 Inode Exhaustion (a Sneaky Variant)

A filesystem can report free *space* via `df -h` but still be “full” because it has run out of **inodes** — the fixed number of metadata slots allocated when the filesystem was created, independent of actual file sizes. This happens when a directory accumulates **millions of tiny files** (a very common cause: a runaway cache, session-file directory, or a logging bug creating one file per request instead of appending).

```
df -i                                # IUse% column — if this is near 100%, that's your problem,
↪ not space
find /path -xdev -printf '.' | wc -c # count files quickly in a suspect directory
```

9.3 11.3 “Deleted” Files Still Consuming Space

If a process still has a file **open** when it’s deleted (`rm`), the kernel keeps the underlying data blocks allocated until the last file descriptor referencing it is closed — `du/l`s won’t show the file anymore (it has no name), but `df` still reports the space as used.

```
lsdf +L1                             # list open files with a link count of 0 — i.e., “deleted”
↪ but still held open”
lsdf /var/log/app.log                 # or specifically check who's holding a given
↪ (now-deleted) file
```

Fix: restart or signal the process holding the file descriptor (e.g., `systemctl restart myapp`, or HUP a logging daemon configured to reopen its log files) so it releases the space.

9.4 11.4 Log Files Consuming All Storage

This is one of the single most common real-world outages. Root causes and fixes: - No logrotate configured for an app that logs heavily → configure rotation (Part 9.3). - An application stuck in a crash-loop logging the same stack trace millions of times per minute → fix the underlying crash, or add rate-limiting/rotation as a stopgap. - Debug-level logging accidentally left on in production → correct the log level.

Common mistakes: - Running `du -sh /` on a live, busy production server without `2>/dev/null` and without limiting depth — it can be slow and noisy on huge filesystems; prefer `du -sh /var/* 2>/dev/null` narrowed down progressively, or `ncdu` (interactive disk usage analyzer) if installed. - Deleting a large log file with `rm` while the writing process still holds it open, expecting immediate space recovery — space is only freed once the process closes/reopens the file (see 11.3). Truncating is safer: `> /var/log/big.log` (empties the file *in place*, same inode, immediate space recovery) instead of `rm`.

10 PART 12 — Memory Management

10.1 12.1 Concepts

- **RAM** — physical memory, fastest storage tier, volatile (lost on power-off).
- **Virtual memory** — the abstraction the kernel gives every process: each process believes it has its own full, contiguous address space, mapped by the kernel (via page tables) onto physical RAM and, when needed, **swap**.
- **Page cache** — the kernel automatically caches recently-read/written disk data in otherwise-unused RAM, to speed up future access. This is why `free -h` “used” memory often looks high on Linux — most of it is *reclaimable* cache, not memory truly locked up by applications. This is normal, healthy behavior, not a leak.
- **Buffers** — memory used for raw block I/O metadata (smaller category, similar reclaimable nature to cache).
- **OOM Killer** — when the kernel genuinely runs out of memory (including swap), it invokes the **Out-Of-Memory killer**, which picks a process (using a heuristic “badness” score, roughly favoring killing large, low-priority processes) and force-kills it to free memory and keep the system alive.

10.2 12.2 Commands

Command	Purpose
<code>free -h</code>	Snapshot of total/used/free RAM and swap, human-readable
<code>vmstat 1</code>	Repeating snapshot of memory, swap, I/O, CPU — great for spotting trends
<code>top/htop</code>	Per-process memory usage, live
<code>ps aux --sort=-%mem</code>	Top memory-consuming processes
<code>sar -r</code>	Historical memory stats (if <code>sysstat</code> is installed and configured)

```
free -h
#              total        used        free        shared  buff/cache   available
# Mem:         15Gi         3.2Gi         8.1Gi         120Mi         4.2Gi         11Gi
```

Read this as: used is what’s genuinely allocated to processes; buff/cache is reclaimable; available is the realistic “how much can a new process actually get” figure — **watch available, not raw free**, when judging memory pressure.

10.3 12.3 Investigating High Memory Usage / OOM Kills

```
dmesg -T | grep -i "killed process"      # confirm the OOM killer actually fired, and
↳ which process/PID it picked
ps aux --sort=-%mem | head -10           # what's using memory right now
journalctl -k | grep -i oom              # kernel journal view of the same event
free -h                                  # is swap also exhausted, or just RAM?
```

DevOps use case: in Kubernetes, a container hitting its memory limit gets OOM-killed by the **cgroup** (Part 26) memory controller — visible as `OOMKilled` in `kubectl describe pod`, the exact same underlying kernel mechanism as a bare-metal OOM kill, just scoped to that container’s cgroup instead of the whole host.

Common mistakes: - Reading free's "used" column in isolation and panicking about "high memory usage" that is actually mostly reclaimable page cache. - Not setting memory limits on containers/services, so one runaway process can exhaust host memory and get everything else on the box OOM-killed too ("noisy neighbor" problem).

11 PART 13 — CPU Management

11.1 13.1 Concepts

- **Core**: a physical processing unit on the CPU chip.
- **Thread** (in this context, a hardware thread / “logical CPU,” e.g. via Hyper-Threading/SMT): lets one physical core present as 2 logical CPUs to the OS scheduler.
- **Context switching**: the kernel scheduler pausing one process/thread and resuming another on a CPU core — necessary since there are usually far more runnable processes than cores, but excessive switching has overhead (“thrashing”).
- **Load average**: a rolling measure of the *number of processes* wanting CPU time (running or waiting, e.g., for disk I/O), averaged over 1, 5, and 15 minutes.
- **I/O wait (%wa)**: percentage of time the CPU was idle *specifically because* it was waiting on a pending disk/network I/O operation — a CPU that looks “idle” with high %wa usually means your bottleneck is **storage**, not CPU.
- **Steal time (%st)**: on virtualized/cloud instances, the percentage of time your VM *wanted* CPU but the hypervisor gave it to another tenant instead — high steal time on an EC2 instance signals you may need a larger/dedicated instance type, not a code-level fix.

11.2 13.2 Understanding Load Average Properly

uptime / top shows something like load average: 2.10, 1.85, 1.50. This is **not a percentage** — it’s the average number of processes wanting a CPU. **You must compare it against your core count**:

```
nproc          # how many logical CPUs you have, e.g. 4
uptime         # load average: 2.10, 1.85, 1.50
```

On a 4-core box, a load average of 2.10 means the system was, on average, about 50% utilized (2.1 of 4 cores worth of demand) — **not overloaded**. A load average of 8.0 on that same 4-core box means real contention — twice as many processes wanting CPU as there are cores to give.

11.3 13.3 Commands

Command	Purpose
top/htop	Live per-process and aggregate CPU view
uptime	Quick load average + how long the box has been up
lscpu	Static CPU info — core count, model, cache sizes, virtualization flags
vmstat 1	Repeating snapshot including %us (user), %sy (system), %id (idle), %wa (I/O wait), %st (steal)
mpstat -P ALL 1	Per-core CPU breakdown (from sysstat package)

```
vmstat 1 5
# procs -----memory----- --swap-- -----io----- -system-- -----cpu-----
#  r  b  swpd  free  buff  cache  si  so  bi  bo  in  cs  us  sy  id  wa  st
```

- High r (runnable) column consistently above your core count → genuine CPU contention.
- High b (blocked) column → processes stuck waiting on I/O.

- High `wa` → disk bottleneck, not CPU.
- Nonzero, sustained `st` on a cloud VM → hypervisor-level contention.

Common mistakes: - Judging load average without checking `nproc` first — “load average 4” means completely different things on a 2-core box (overloaded) vs a 32-core box (barely used).
 - Treating high `%wa` as a CPU problem and trying to “add more CPU” — the fix is almost always disk/storage-side (faster disk, less I/O, caching), not CPU. # PART 14 — Linux Networking

This is the part DevOps engineers use daily — misconfigured networking is behind the majority of “it works locally but not in prod” and “I can’t reach the server” incidents.

11.4 14.1 OSI Model and TCP/IP Model

OSI Layer	TCP/IP Layer	Examples
7 Application	Application	HTTP, HTTPS, DNS, SSH
6 Presentation	Application	TLS
5 Session	Application	—
4 Transport	Transport	TCP, UDP
3 Network	Internet	IP, routing
2 Data Link	Link	Ethernet, MAC addresses, ARP
1 Physical	Link	Cables, radio (Wi-Fi)

DevOps troubleshooting is almost always about working through these layers bottom-up (or top-down) to find where the chain breaks.

11.5 14.2 Addressing

- **MAC address:** a hardware address burned into (or virtually assigned to) a network interface, used for **local network (layer 2)** delivery.
- **IP address:** a logical address for **routing across networks (layer 3)**. **IPv4** — 32-bit, dotted-decimal (10.0.1.15), the address space DevOps engineers deal with daily. **IPv6** — 128-bit, hex-colon notation, increasingly common but often still secondary in typical cloud VPC setups.
- **CIDR** notation (10.0.1.0/24) — an IP range: the number after / is how many leading bits are the fixed **network** portion; the rest is the **host** portion. /24 = 256 addresses (254 usable), /16 = 65,536 addresses. This is exactly how AWS VPC/subnet sizing works.
- **Subnet:** a logical subdivision of a network — e.g., a VPC (10.0.0.0/16) split into a public subnet (10.0.1.0/24) and private subnet (10.0.2.0/24).
- **Gateway:** the router an interface sends traffic to when the destination isn’t on its local subnet — “the door out of this network.”

11.6 14.3 DNS and DHCP

- **DNS** (Domain Name System) translates human-readable names (example.com) into IP addresses, via a hierarchy of resolvers and authoritative nameservers.
- **DHCP** (Dynamic Host Configuration Protocol) automatically assigns an IP address, gateway, and DNS servers to a device when it joins a network — this is how most cloud instances get their private IP without manual configuration.

11.7 14.4 ARP

ARP (Address Resolution Protocol) maps an IP address to a MAC address **within the local network segment** — before host A can send a packet to host B on the same subnet, it needs B's MAC address, discovered via an ARP broadcast/reply.

11.8 14.5 TCP vs UDP, Ports, Sockets

- **TCP**: connection-oriented, reliable, ordered, with handshakes and retransmission — used by HTTP/HTTPS, SSH, most application traffic.
- **UDP**: connectionless, no delivery guarantee, lower overhead — used by DNS queries, video/voice streaming, some monitoring protocols.
- **Port**: a 16-bit number identifying *which application/service* on a host a connection is for (e.g., 22=SSH, 80=HTTP, 443=HTTPS, 5432=PostgreSQL).
- **Socket**: the combination of (protocol, source IP, source port, destination IP, destination port) uniquely identifying one connection/communication endpoint.

11.9 14.6 HTTP, HTTPS, TLS, SSH

- **HTTP**: application-layer protocol for web traffic, plaintext.
- **HTTPS**: HTTP **wrapped in TLS** — encrypts and authenticates the connection.
- **TLS** (Transport Layer Security): the protocol that provides that encryption/authentication (successor to SSL) — handles the certificate handshake, key exchange, and encrypted channel.
- **SSH** (Secure Shell): an encrypted protocol for remote login and command execution — covered in depth in Part 15.

11.10 14.7 NAT, Proxies, Load Balancers

- **NAT** (Network Address Translation): rewrites source/destination IPs, most commonly to let many private-IP hosts share one public IP for outbound traffic — this is exactly how an AWS **NAT Gateway** lets instances in a private subnet reach the internet without themselves having a public IP.
- **Proxy**: an intermediary that forwards requests **on behalf of a client** (a forward proxy) — e.g., a corporate proxy controlling/logging outbound traffic.
- **Reverse proxy**: sits in front of one or more backend servers and forwards **incoming** client requests to them, often adding caching, TLS termination, or routing logic (Nginx and HAProxy are classic examples).
- **Load balancer**: distributes incoming traffic across multiple backend instances for scalability and fault tolerance (AWS ALB/NLB, Nginx, HAProxy).

11.11 14.8 Networking Commands

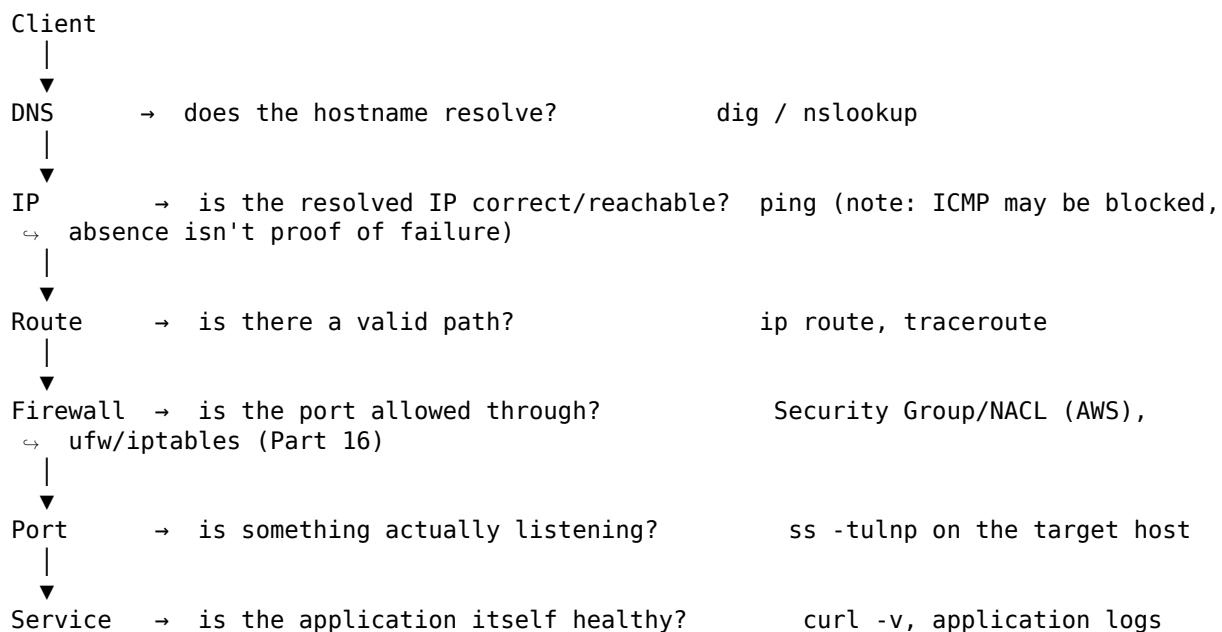
Command	Purpose	Example
<code>ip addr</code>	Show IP addresses on interfaces	<code>ip addr show</code>
<code>ip link</code>	Show/manage network interfaces (up/down)	<code>sudo ip link set eth0 up</code>
<code>ip route</code>	Show/manage the routing table	<code>ip route show</code>
<code>ping</code>	Test reachability via ICMP	<code>ping -c 4 8.8.8.8</code>

Command	Purpose	Example
tracert	Show the route (hop-by-hop) to a destination	tracert example.com
tracert	Similar to tracert, no root needed	tracert example.com
ss	Show sockets/listening ports (modern replacement for netstat)	ss -tulnp
netstat	Older socket/connection viewer	netstat -tulnp
dig	DNS lookup, detailed	dig example.com +short
nslookup	Simpler DNS lookup	nslookup example.com
host	Minimal DNS lookup	host example.com
curl	Make HTTP(S)/other requests from the CLI	curl -I https://example.com
wget	Download files over HTTP(S)/FTP	wget https://example.com/file.tar.gz
nc (netcat)	Raw TCP/UDP connections — “network Swiss army knife”	nc -zv host 443 (port check)
telnet	Legacy raw TCP client (still handy for port testing)	telnet host 80
arp	Show/manage the ARP cache	arp -a
tcpdump	Capture and inspect raw packets	sudo tcpdump -i eth0 port 443

```
ss -tulnp
# -t TCP, -u UDP, -l listening only, -n numeric (skip DNS), -p show owning process
```

This one command answers “what is actually listening on this box, and on which port, owned by which process” — one of the most-used commands during any connectivity incident.

11.12 14.9 Practical Troubleshooting: The Full Chain



dig example.com +short	# 1. DNS resolving?
ping -c 3 <resolved-ip>	# 2. Host reachable at all? (may be blocked –
↳ not conclusive alone)	
tracert <resolved-ip>	# 3. Where does the path break, if anywhere?
curl -v telnet://<ip>:443	# 4/5. Can I even open the TCP port? (or: nc
↳ -zv ip 443)	
curl -v https://example.com	# 6. Full app-layer response – what does the
↳ server actually say?	
ss -tulnp grep :443	# (on the server itself) is anything
↳ actually bound to that port?	

Common mistakes: assuming a failed ping proves the host is down — many production hosts/security groups intentionally block ICMP while still serving TCP traffic perfectly fine; always confirm with a port-level check (nc/curl) before concluding “the server is down.”

12 PART 15 — SSH

12.1 15.1 Core Commands

Command	Purpose
ssh user@host	Open a remote shell
scp file user@host:/path	Copy files over SSH
sftp user@host	Interactive file transfer over SSH
ssh-keygen	Generate a public/private key pair
ssh-agent	Background process that holds decrypted private keys in memory for the session
ssh-add	Add a key to the running ssh-agent

12.2 15.2 Public/Private Key Authentication

```
ssh-keygen -t ed25519 -C "aryan@laptop"    # generates ~/.ssh/id_ed25519 (private) and
↳ .pub (public)
ssh-copy-id user@host                      # appends your public key to the remote's
↳ authorized_keys
ssh user@host                             # now authenticates without a password
```

- **Private key** — stays on your machine, never shared, must be chmod 600.
- **Public key** — safe to share; goes into ~/.ssh/authorized_keys on every server you want to access.
- **~/.ssh/known_hosts** — caches the server's public key fingerprint the first time you connect, so SSH can warn you if it ever changes unexpectedly (a potential man-in-the-middle indicator, or just a legitimately re-provisioned server).

12.3 15.3 Configuration Files

- **~/.ssh/config** (client-side): define shortcuts/defaults per host.

```
Host prod-web
  HostName 10.0.1.25
  User ubuntu
  IdentityFile ~/.ssh/prod_key.pem
  Port 22
```

Then just ssh prod-web.

- **/etc/ssh/sshd_config** (server-side daemon config) — key hardening directives:

```
PasswordAuthentication no    # disable password login entirely — keys only
PermitRootLogin no           # don't allow direct root SSH login
Port 2222                    # optional: change from default 22 (mild obscurity,
↳ not real security)
AllowUsers deploy aryan      # explicit allowlist of who can SSH in at all
```

After editing: `sudo systemctl restart sshd` (and **keep your current session open** while testing a new one — a config mistake can lock you out).

12.4 15.4 SSH Hardening Checklist

- Disable password authentication; keys only.
- Disable root login (`PermitRootLogin no`); use a named user + `sudo`.

- Keep authorized_keys and private keys at 600/700 permissions.
- Use AllowUsers/AllowGroups to restrict who can connect.
- Consider changing the default port (reduces automated scan noise, not a real security boundary on its own).
- Use fail2ban (Part 21) to auto-block repeated failed login attempts.
- Prefer short-lived credentials/IAM-based SSH (e.g., AWS EC2 Instance Connect, SSM Session Manager) over long-lived static keys where possible.

12.5 15.5 Port Forwarding

- **Local forwarding** (ssh -L): forward a port on *your* machine to a service reachable from the remote host — e.g., reach a private RDS database through a bastion:

```
ssh -L 5432:rds-endpoint.internal:5432 user@bastion-host
# now localhost:5432 on YOUR machine tunnels to the DB, through the bastion
```

- **Remote forwarding** (ssh -R): the reverse — expose a port on *your* local machine to the remote server (useful for webhook testing, exposing a local dev server).
- **Jump host** (ssh -J): SSH through an intermediate bastion to reach a host with no direct public route — very common for reaching private-subnet EC2 instances.

```
ssh -J bastion-user@bastion-host app-user@10.0.2.15
```

12.6 15.6 Connecting to AWS EC2 Securely

```
chmod 400 my-ec2-key.pem # AWS keys must be strictly private
ssh -i my-ec2-key.pem ubuntu@<ec2-public-ip> # default user: 'ubuntu' (Ubuntu AMIs)
↪ or 'ec2-user' (Amazon Linux)
```

Best practice for production: prefer **SSM Session Manager** (no open inbound port 22 at all, IAM-authenticated, fully audited) over direct key-based SSH where your setup allows it.

Common mistakes: - .pem/private key permissions too open (644) — SSH refuses with "UN-PROTECTED PRIVATE KEY FILE". Fix: chmod 400 or 600. - Testing an sshd_config hardening change by restarting sshd and *immediately closing your only session* — if the config is broken, you're now locked out. Always keep the current session open and verify a **new** connection works before closing the old one.

13 PART 16 — Firewalls

13.1 16.1 iptables, nftables, and ufw

- **iptables**: the classic Linux firewall tool, manipulating kernel **netfilter** rules directly — powerful but low-level, rule-ordering-sensitive syntax.
- **nftables**: the modern successor to iptables, same underlying netfilter framework, cleaner syntax, gradually replacing iptables as the default backend on newer distros.
- **ufw** (Uncomplicated Firewall): a friendly front-end over iptables/nftables, the default on Ubuntu — this is what you'll actually type day-to-day.

13.2 16.2 Practical UFW Examples

```
sudo ufw status verbose           # check current state and rules
sudo ufw default deny incoming    # secure baseline: block everything inbound by
↳ default
sudo ufw default allow outgoing    # allow all outbound by default (typical starting
↳ point)
sudo ufw allow 22/tcp              # allow SSH
sudo ufw allow 80/tcp              # allow HTTP
sudo ufw allow 443/tcp            # allow HTTPS
sudo ufw allow from 10.0.0.0/24 to any port 5432 # allow Postgres only from a
↳ specific internal subnet
sudo ufw enable                   # activate ([!]) do this AFTER allowing SSH,
↳ or you'll lock yourself out!)
sudo ufw delete allow 80/tcp       # remove a rule
```

[DANGEROUS]: enabling ufw with default deny incoming **before** explicitly allowing your SSH port will immediately cut your own remote session and lock you out of a headless/cloud server (recoverable only via console access, if available). Always allow SSH first.

13.3 16.3 Linux Firewall vs AWS Security Groups/NACLs

	Host firewall (ufw/iptables)	AWS Security Group	AWS NACL
Where it lives	On the instance's own kernel	At the AWS hypervisor/network layer, attached to ENIs	At the subnet boundary
State	Stateful (by default with ufw/most iptables setups)	Stateful (return traffic auto-allowed)	Stateless (must explicitly allow both directions)
Scope	That one instance only	One or more instances (via ENI attachment)	Every instance in the subnet
Default	Depends on config	Deny all inbound by default, allow all outbound	Allow all by default (default NACL)

Practical implication: in AWS, network traffic is filtered at *multiple layers* — Security Group, then NACL, then (optionally) the instance's own ufw/iptables. A connectivity issue can be

caused by **any** of these three, independently — troubleshooting means checking all three, not just one.

14 PART 17 — Package Management

14.1 17.1 APT (Debian/Ubuntu)

Command	Purpose
apt update	Refresh the local package index from configured repositories (does NOT install/upgrade anything itself)
apt upgrade	Upgrade all installed packages to latest available versions
apt install <pkg>	Install a package (and its dependencies)
apt remove <pkg>	Remove a package, keep its config files
apt purge <pkg>	Remove a package and its config files
apt autoremove	Remove packages that were installed as dependencies and are no longer needed
apt search <term>	Search available packages
apt-cache show <pkg>	Show detailed metadata about a package
dpkg -i file.deb	Install a local .deb package file directly
dpkg -l	List all installed packages

```
sudo apt update && sudo apt upgrade -y      # standard "get current" combo – note the
↪ &&, always update first
sudo apt install -y nginx
sudo apt list --installed | grep nginx
```

Concepts: a **repository** is a remote server hosting packages + an index (/etc/apt/sources.list, /etc/apt/sources.list.d/); APT resolves **dependencies** automatically (installing a package's required libraries too); **security updates** are just regular package upgrades from the distro's security repository — running apt update && apt upgrade regularly is basic security hygiene.

14.2 17.2 YUM / DNF / RPM (RHEL, CentOS, Rocky, AlmaLinux, Amazon Linux)

Command	Purpose
dnf install <pkg>	Install (DNF — modern successor to YUM, used on RHEL 8+/Rocky/Alma/Amazon Linux 2023)
yum install <pkg>	Install (YUM — older, RHEL 7/Amazon Linux 2, syntax mostly interchangeable with dnf)
dnf update	Update all packages (combines “refresh index” + “upgrade” into one, unlike apt's two-step)
dnf remove <pkg>	Remove a package
dnf search <term>	Search available packages
rpm -ivh file.rpm	Install a local .rpm file directly
rpm -qa	List all installed packages

```
sudo dnf update -y
sudo dnf install -y nginx
```

Common mistakes: - Running apt upgrade without apt update first — you'll upgrade against a stale index and may miss the latest available versions. - Installing software by manually downloading and compiling from source when a package manager version exists — loses automatic security updates and dependency tracking; prefer the package manager (or an official Docker image) unless you have a specific reason not to. - Mixing apt/dpkg package installs with manually-placed files in the same directories a package manages — causes conflicts on the next upgrade. # PART 18 — Environment Variables

14.3 18.1 Core Variables

Variable	Meaning
PATH	Colon-separated list of directories the shell searches, in order, to find a command's executable
HOME	Current user's home directory
USER	Current username
SHELL	Path to the user's default login shell
PWD	Current working directory (kept in sync by the shell)

```
echo $PATH
# /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

14.4 18.2 Commands

Command	Purpose
env	Show/run a command with a modified environment
printenv	Print environment variables
export VAR=value	Set a variable and make it available to child processes
unset VAR	Remove a variable
echo \$VAR	Print a variable's value

```
export DB_HOST=10.0.1.5      # available to any command/script launched from THIS
⇨ shell onward
NODE_ENV=production node app.js  # set a variable for just this one command's
⇨ invocation
```

Key distinction: a plain shell variable (VAR=value) is only visible to the current shell. export VAR=value promotes it to the **environment**, meaning any child process (a script, another program you launch) inherits a copy of it too. This is exactly how apps read config like DATABASE_URL or AWS_REGION.

14.5 18.3 Where Variables Get Set: .bashrc, .profile, /etc/environment

File	Scope	When it runs
~/.bashrc	Per-user	Every interactive non-login shell (e.g., opening a new terminal tab, or most SSH sessions)
~/.profile (or ~/.bash_profile)	Per-user	Login shells (console login, su -, some SSH clients)
/etc/environment	System-wide	Set at login, simple KEY=value pairs only (no shell logic allowed)
/etc/profile, /etc/profile.d/*.sh	System-wide	Login shells, for all users

Common mistakes: putting an export in ~/.bashrc and then being confused why a **non-interactive** script or a cron job doesn't see it — cron and many automation tools don't source .bashrc at all; use /etc/environment, an explicit source in the script, or set the variable directly in the systemd unit/crontab entry instead.

14.6 18.4 PATH Resolution and Command Lookup

When you type a bare command name (nginx, python3), the shell searches each directory in \$PATH, **in order**, for a matching executable file, and runs the **first** match found.

```
which python3          # shows the full path that WOULD be executed
type python3           # similar, also reveals if it's a shell builtin/alias instead
↪ of a real file
```

DevOps gotcha: multiple versions of a tool installed in different directories (e.g., system Python vs a pyenv-managed Python) — PATH order determines which one actually runs, a very common source of “works on my machine, different version in CI” bugs. Always verify with which/type when debugging version mismatches.

15 PART 19 — Bash Shell Scripting

15.1 19.1 Variables, Strings, Numbers, Arrays

```
#!/bin/bash
name="Aryan"                # no spaces around =
count=5
echo "Hello, $name"         # variable expansion inside double quotes
echo 'Hello, $name'        # single quotes: NO expansion, literal text

arr=(web1 web2 web3)        # array
echo "${arr[0]}"            # web1
echo "${arr[@]}"            # all elements
echo "${#arr[@]}"           # array length (3)

result=$((count + 10))      # arithmetic — must use $(( ))
```

15.2 19.2 Conditionals

```
if [ "$count" -gt 3 ]; then
    echo "count is greater than 3"
elif [ "$count" -eq 3 ]; then
    echo "count is exactly 3"
else
    echo "count is 3 or less"
fi

# [[ ]] is the modern, safer bash-specific test — supports regex, no word-splitting
↪ surprises
if [[ "$name" =~ ^A ]]; then
    echo "Name starts with A"
fi

case "$1" in
    start) echo "starting" ;;
    stop) echo "stopping" ;;
    *) echo "unknown command" ;;
esac
```

[] is the POSIX test command (portable, but fragile with unquoted variables/special characters). [[]] is a bash keyword — prefer it in bash scripts for safety and extra features (regex match with =~, no need to quote as defensively).

15.3 19.3 Loops

```
for host in web1 web2 web3; do
    echo "Deploying to $host"
done

for i in {1..5}; do echo "$i"; done

while read -r line; do
    echo "Log line: $line"
done < app.log
```

```
until systemctl is-active --quiet myapp; do
    echo "Waiting for myapp to start..."
    sleep 2
done
```

15.4 19.4 Functions, Arguments, Exit Codes

```
deploy() {
    local env="$1"           # 'local' scopes the variable to the function – good practice
    local version="$2"
    echo "Deploying version $version to $env"
}
deploy "production" "1.4.2"

echo "Script name: $0"
echo "First arg: $1"
echo "All args: $@"
echo "Arg count: $#"
```

deploy prod 1.4.2

```
echo "Exit code of last command: $?"      # every command returns an exit code: 0 =
↪ success, nonzero = failure
```

15.5 19.5 Command Substitution, Arithmetic, Tests

```
now=$(date +%F)                # preferred modern syntax
files=$(ls /var/log | wc -l)

total=$((5 + 3 * 2))
```

```
if [ -f /etc/nginx/nginx.conf ]; then echo "config exists"; fi  # -f: regular file
↪ exists
if [ -d /var/log/myapp ]; then echo "dir exists"; fi           # -d: directory exists
if [ -z "$var" ]; then echo "var is empty"; fi                 # -z: string is empty
if [ -x script.sh ]; then echo "executable"; fi                # -x: file is
↪ executable
```

15.6 19.6 Traps and Signals in Scripts

```
cleanup() {
    echo "Cleaning up temp files..."
    rm -f /tmp/deploy_lock
}
trap cleanup EXIT SIGTERM SIGINT      # run cleanup on normal exit OR on receiving
↪ SIGTERM/SIGINT
```

This pattern — trapping EXIT/SIGTERM to release a lock file, close a connection, or roll back a partial change — is essential in real deployment scripts.

15.7 19.7 A Real DevOps Script: Health-Check + Restart

```
#!/bin/bash
set -euo pipefail    # exit on error, error on unset variables, fail on any pipeline
↳ stage error – always use this

APP_URL="http://localhost:8080/health"
SERVICE_NAME="myapp"
LOG_FILE="/var/log/health-check.log"

log() {
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $1" >> "$LOG_FILE"
}

status_code=$(curl -s -o /dev/null -w "%{http_code}" "$APP_URL" || echo "000")

if [ "$status_code" -ne 200 ]; then
    log "Health check FAILED (status: $status_code). Restarting $SERVICE_NAME."
    systemctl restart "$SERVICE_NAME"
    sleep 5
    new_status=$(curl -s -o /dev/null -w "%{http_code}" "$APP_URL" || echo "000")
    if [ "$new_status" -eq 200 ]; then
        log "Restart successful, service healthy again."
    else
        log "Restart did NOT fix the issue – status: $new_status. Manual intervention
        ↳ needed."
    fi
else
    log "Health check OK (status: $status_code)."
fi
```

set -euo pipefail is close to mandatory for production bash scripts: -e stops the script at the first failing command instead of plowing ahead with a broken state, -u catches typos in variable names (referencing an unset variable is an error, not silently empty), -o pipefail makes a pipeline fail if **any** stage fails, not just the last one.

Common mistakes: - Forgetting to quote variables (rm -rf \$DIR/* instead of rm -rf "\$DIR"/*) — breaks on paths with spaces and is a classic source of accidental wildcard/word-splitting disasters. - Not using set -euo pipefail, so a failed command (e.g., a failed cd) is silently ignored and the rest of the script runs against the wrong directory. - Comparing strings with -eq/-gt (numeric operators) instead of != — will error out or behave unexpectedly on non-numeric input.

16 PART 20 — Cron & Automation

16.1 20.1 cron and crontab

cron is a time-based job scheduler daemon. Each user has their own **crontab** (cron table) defining recurring jobs.

```
crontab -e      # edit your own crontab
crontab -l      # list your current crontab
crontab -r      # remove your entire crontab (!!) no confirmation!
```

16.1.1 Crontab syntax

```
# |----- minute (0-59)
# |----- hour (0-23)
# |----- day of month (1-31)
# |----- month (1-12)
# |----- day of week (0-6, Sun=0)
# |
# * * * * * command-to-run

0 2 * * * /opt/scripts/backup.sh           # every day at 2:00 AM
*/15 * * * * /opt/scripts/health-check.sh   # every 15 minutes
0 0 * * 0 /opt/scripts/weekly-report.sh     # every Sunday at midnight
0 3 1 * * /opt/scripts/monthly-cleanup.sh   # 1st of every month at
↳ 3 AM
```

Best practice: always redirect output and use absolute paths in cron jobs — cron runs with a minimal environment (no \$PATH from your interactive shell, no working directory guarantee):

```
0 2 * * * /usr/bin/flock -n /tmp/backup.lock /opt/scripts/backup.sh >>
↳ /var/log/backup.log 2>&1
```

(flock here prevents a second run from overlapping if the previous backup is still running.)

16.2 20.2 at — One-Time Scheduled Jobs

```
at 22:00 today
at> /opt/scripts/deploy.sh
at> <Ctrl+D>
```

```
atq          # list pending at jobs
atrm <job-id> # cancel one
```

Use at for a **one-off** scheduled task; use cron for anything **recurring**.

16.3 20.3 systemd Timers (Modern Alternative to cron)

```
# /etc/systemd/system/backup.timer
[Unit]
Description=Run backup daily

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
```

```
[Install]
WantedBy=timers.target
```

```
# /etc/systemd/system/backup.service
```

```
[Unit]
Description=Backup job
```

```
[Service]
Type=oneshot
ExecStart=/opt/scripts/backup.sh
```

```
sudo systemctl enable --now backup.timer
systemctl list-timers # see all scheduled timers and their next run
```

Advantages over cron: logs go straight to `journalctl -u backup.service` (no manual log redirection needed), `Persistent=true` automatically catches up a missed run if the machine was off, and dependency management (`After=`, `Requires=`) integrates with the rest of `systemd`.

16.4 20.4 Practical Automation Examples

```
# Backup: dump a database and rotate old backups, keep 7 days
0 1 * * * pg_dump mydb | gzip > /backups/mydb-$(date +%F).sql.gz && find /backups
↪ -mtime +7 -delete
```

```
# Log cleanup
0 3 * * * find /var/log/myapp -name "*.log" -mtime +14 -delete
```

```
# Health check + alert (calling out to a script that pages on failure)
*/5 * * * * /opt/scripts/health-check.sh
```

```
# Automated report
0 8 * * 1 /opt/scripts/weekly-report.sh | mail -s "Weekly Report" team@company.com
```

Note: `%` has special meaning in crontab (newline) — it must be escaped as `\%` when used inside a command, e.g., in a date format string.

Common mistakes: - Writing a cron command that works fine when you run it manually but fails under cron — almost always a `$PATH`/environment difference; always use absolute paths for both the script and any binaries it calls, and test with `env -i` to simulate cron's minimal environment. - No overlap protection on a job that might occasionally run longer than its interval (e.g., a 5-minute job that occasionally takes 6 minutes) — leads to multiple instances stacking up; use `flock` or a lock file. - Silent failures — a cron job with no output redirection fails invisibly; always log both `stdout` and `stderr` somewhere you'll actually check, or wire up alerting on failure. # PART 21 — Linux Security

16.5 21.1 Principle of Least Privilege

The foundational security principle: every user, process, and service should have **only** the permissions it needs to do its job, and nothing more. In practice: non-root service users, minimal `sudo` grants, tight file permissions (Part 5), scoped IAM roles on cloud resources.

16.6 21.2 Core Building Blocks (Recap)

- **sudo** — controlled privilege escalation with an audit trail, instead of shared root logins.

- **File permissions / ACLs** — restrict who can read/write/execute what (Part 5).
- **SSH hardening** — key-only auth, no root login, restricted user list (Part 15).
- **Firewall** — restrict network exposure to only necessary ports (Part 16).
- **Users and groups** — proper separation of duties, service accounts per application.

16.7 21.3 SELinux vs AppArmor

Both are **Mandatory Access Control (MAC)** systems — an extra security layer *on top of* standard user/group/permission checks (**Discretionary Access Control**). Even if a process technically has file permission to do something, MAC can still block it based on a security policy.

	SELinux	AppArmor
Used by default on	RHEL, CentOS, Rocky, AlmaLinux, Amazon Linux	Ubuntu, Debian
Policy model	Label-based (every file/process gets a security context); very granular, steeper learning curve	Path-based (profiles tied to file paths); simpler to read and write
Common commands	getenforce, setenforce, semanage, ausearch	aa-status, aa-enforce, aa-complain

```
getenforce          # SELinux: Enforcing / Permissive / Disabled
sudo aa-status      # AppArmor: list of loaded profiles and their mode
```

DevOps relevance: a shockingly common cause of “works everywhere except this one server” is SELinux/AppArmor silently blocking an action (e.g., Nginx unable to proxy to a backend port, or a container denied access to a mounted volume) — always check `audit.log` (`ausearch -m avc -ts recent` on SELinux systems) or `dmesg/journal` for AppArmor denials before concluding it’s a permissions or application bug.

16.8 21.4 Secrets and Password Policy

- **Never** hardcode secrets (DB passwords, API keys) in scripts, config files committed to git, or environment variables baked into a Docker image layer — use a secrets manager (AWS Secrets Manager, HashiCorp Vault, SSM Parameter Store) or injected runtime environment variables instead.
- Password policy (`/etc/login.defs`, `chage` command) controls expiry, minimum length/complexity — increasingly less relevant for servers where **key-based SSH** and **short-lived credentials** replace passwords entirely.

16.9 21.5 Security Updates and Audit Logs

```
sudo apt update && sudo apt upgrade -y      # Debian/Ubuntu — apply pending
↪ security patches
sudo dnf update -y                          # RHEL family
sudo apt install unattended-upgrades        # Ubuntu: automatic security
↪ patching
```

- **Audit logs:** the Linux Audit subsystem (`auditd`) can log security-relevant events (file access, privilege escalation, command execution) beyond what standard app logs capture — used heavily in regulated/compliance environments.

16.10 21.6 Fail2ban

fail2ban watches log files (e.g., /var/log/auth.log) for patterns indicating brute-force attempts (repeated failed SSH logins) and automatically adds a temporary firewall ban for the offending IP.

```
sudo apt install fail2ban
sudo systemctl enable --now fail2ban
sudo fail2ban-client status sshd      # see currently banned IPs for the SSH jail
```

Common mistakes: relying on SSH port obscurity or fail2ban *alone* as your security model — these are defense-in-depth layers, not substitutes for key-only auth, least privilege, and a properly scoped firewall/Security Group.

17 PART 22 — Linux Performance

17.1 22.1 The Systematic Troubleshooting Framework

Rather than guessing, work through resources in order — most performance problems originate at one of these layers and cascade to look like problems in the layers above:

CPU → Memory → Disk → Network → Application

Layer	Symptom	First commands
CPU	High load average relative to nproc, slow response times	top, vmstat 1, mpstat -P ALL 1
Memory	Swapping, OOM kills, slow due to reclaiming cache	free -h, vmstat 1, dmesg \ grep -i oom
Disk I/O	High %wa, high iowait, slow writes/reads	iostat -x 1, iotop, df -h
Network	Latency, timeouts, packet loss	ss -s, ping, tcpdump, traceroute
Application	The app itself is slow/leaking regardless of healthy infra	Application-level profiling, slow query logs, strace

17.2 22.2 Tools

Tool	Purpose
top/htop vmstat iostat -x 1	Live overview across CPU/memory/processes Memory, swap, I/O, CPU summary, repeating Per-device disk I/O stats — %util, await (latency) are the key columns
iotop sar	Per-process disk I/O, like top but for disk Historical performance data (requires sysstat, useful for “what happened at 3 AM last night”)
ss -s lsof	Socket statistics summary List open files/sockets — “what is this process actually holding open”
strace	Trace syscalls a process makes — deep debugging (Part 23)

```
iostat -x 1 5
# Key columns: %util (how busy the device is), await (avg I/O latency in ms), r/s, w/s
↪ (throughput)
# %util consistently near 100% with rising await = the disk itself is your bottleneck
```

17.3 22.3 Diagnosing Each Bottleneck Type

- **CPU bottleneck:** load average significantly above nproc, high %us/%sy in top, specific process pegged at high %CPU. Fix: optimize code, add caching, scale horizontally, or increase instance size.
- **Memory bottleneck:** free -h shows near-zero available, si/so (swap in/out) nonzero and climbing in vmstat, OOM kills in dmesg. Fix: fix a leak, add memory limits, scale up, or add caching layer tuning.

- **Disk I/O bottleneck:** high %wa in top/vmstat, high await/%util in iostat. Fix: faster storage (e.g., move from gp2 to gp3/io2 on AWS), reduce write amplification, add read caching, review database indexing.
- **Network bottleneck:** high latency in ping/app-level timing, packet loss visible with ping/mtr, ss -s showing many retransmits. Fix: check for saturated bandwidth, misconfigured MTU, distance/routing issues, or an overloaded upstream service.
- **Process/application bottleneck:** infra all looks healthy, but the app itself is slow — profile the code, check slow DB queries, check for lock contention, thread pool exhaustion.

Common mistakes: optimizing the wrong layer because a downstream symptom masks the real bottleneck — e.g., “the app is slow” (application-layer symptom) when the true root cause is disk I/O saturation from another process on the same host (a classic noisy-neighbor problem, especially without cgroup limits).

18 PART 23 — Advanced Linux Tools

Tool	Purpose	Example / DevOps use case
lsof	List open files (including sockets, pipes) held by processes	<code>lsof -i :8080</code> — find what's bound to a port; <code>lsof +L1</code> — find deleted-but-open files (Part 11.3)
strace	Trace system calls a process makes in real time	<code>strace -f -e trace=network -p <PID></code> — see exactly what network calls a stuck process is making
ltrace	Trace library calls (higher level than syscalls)	<code>ltrace ./mybinary</code> — debug what library functions a program is calling
dmesg	Kernel ring buffer messages	<code>dmesg -T \ tail -50</code> — hardware, driver, and OOM events
sysctl	Read/set kernel runtime parameters	<code>sysctl net.ipv4.ip_forward / sudo sysctl -w net.core.somaxconn=1024</code>
ulimit	View/set per-process resource limits (open files, processes, memory)	<code>ulimit -n 65536</code> — raise open-file-descriptor limit for a high-connection-count service
watch	Re-run a command repeatedly, showing live-updating output	<code>watch -n 1 'df -h'</code> — watch disk fill up in real time
screen	Terminal multiplexer — sessions survive disconnection	<code>screen -S mysession</code> then detach with <code>Ctrl+A D</code> , reattach with <code>screen -r mysession</code>
tmux	Modern terminal multiplexer, panes/windows, scriptable	<code>tmux new -s deploy</code> — keep a long-running deploy/build alive across an SSH drop

```

strace -c ls /                                # summarize syscall time/count breakdown for a quick
↪ command
strace -f -p 1234 -o trace.log                # attach to a running process (including its forked
↪ children with -f), log to file
lsof -i :443                                  # what process is bound to port 443?
lsof -u appuser                               # everything a specific user's processes have open
sysctl -a | grep somaxconn                    # inspect a specific kernel networking tuning
↪ parameter

```

DevOps use case for screen/tmux: running a long database migration or build over SSH — if your SSH connection drops (laptop sleeps, network blip), a process started directly in that shell dies with it. Inside tmux/screen, the session (and the process running inside it) **survives** the disconnect; you just reattach later and it's still running. This is one of the most important habits for anyone doing long-running remote work.

Common mistakes: - Running long jobs directly over a bare SSH session without

tmux/screen/nohup — losing the connection kills the job. - Reaching for strace on a production process without -c or filtering (-e trace=...) — it can generate enormous output and measurably slow the traced process; scope it. - Not raising ulimit -n for high-concurrency services (databases, proxies) — default open-file limits (often 1024) are far too low for production load and cause mysterious “too many open files” errors under traffic.
PART 24 — The Kernel (DevOps-Relevant Depth)

The kernel is the single program running in kernel space (Part 1.5) that provides every other process with CPU scheduling, memory management, device access, networking, and filesystems — via the system call interface. For DevOps purposes, you don’t need OS-textbook depth; you need to know **what the kernel is responsible for** and **how to inspect/tune it**, because containers, Kubernetes, and cloud performance all sit directly on top of these mechanisms.

18.1 24.1 What the Kernel Actually Manages

- **Scheduling** — decides which runnable process/thread gets CPU time on which core, and for how long, using a scheduling algorithm (Completely Fair Scheduler by default on Linux).
- **Memory management** — virtual memory, paging, the page cache, and OOM handling (Part 12).
- **Networking stack** — full TCP/IP implementation, from the NIC driver up through sockets that applications use.
- **Filesystems** — implementations of ext4, XFS, btrfs, plus virtual filesystems like /proc and /sys.
- **Device drivers** — code that talks to specific hardware (disks, NICs, GPUs).
- **Kernel modules** — pieces of kernel code that can be loaded/unloaded at runtime without a reboot (e.g., a specific filesystem driver, a network driver).

18.2 24.2 Kernel Modules

```
lsmod                                # list currently loaded kernel modules
sudo modprobe <module>              # load a module (and its dependencies)
sudo modprobe -r <module>           # unload a module
modinfo <module>                    # show details about a module
```

DevOps relevance: container networking (overlay, veth, bridge) and storage drivers (overlay2 for Docker) rely on specific kernel modules being present/loaded — a stripped-down or unusual kernel missing the right module is a real (if less common today) source of “Docker won’t start” issues.

18.3 24.3 sysctl — Kernel Runtime Tuning

```
sysctl -a | less                     # view ALL current kernel parameters
sysctl net.ipv4.ip_forward           # view one specific parameter
sudo sysctl -w net.core.somaxconn=4096 # change temporarily (lost on reboot)
```

Persist changes in /etc/sysctl.conf or /etc/sysctl.d/*.conf, then sudo sysctl -p to apply.

Common production tuning examples:

```
net.ipv4.ip_forward = 1              # required for a host to route/forward packets
↪ (e.g. Docker networking)
net.core.somaxconn = 4096            # increase max queued connections for busy servers
```

```
vm.swappiness = 10                # reduce the kernel's tendency to swap (lower =  
  ↳ prefer RAM, avoid swap)  
fs.file-max = 2097152            # system-wide max open file descriptors
```

18.4 24.4 uname and dmesg

```
uname -a          # kernel name, version, architecture – always useful when  
  ↳ reporting/diagnosing issues  
dmesg -T          # kernel ring buffer with human timestamps (Part 9/12)
```

19 PART 25 — /proc and /sys

19.1 25.1 What These Are

/proc and /sys are **virtual filesystems** — they don't exist on disk at all. The kernel generates their content live, on read, directly from its own internal data structures. This is the **primary mechanism** every monitoring tool (top, ps, CloudWatch agent, Prometheus node_exporter) uses to read system state — they are, under the hood, reading (and sometimes writing) files under /proc and /sys.

19.2 25.2 Practical /proc Files

File	Contains
/proc/cpuinfo	Detailed info on every CPU core
/proc/meminfo	Detailed memory stats — the source free parses
/proc/loadavg	The raw load average numbers uptime/top display
/proc/uptime	System uptime in seconds
/proc/[PID]/	A directory per running process , containing its status, open file descriptors (fd/), memory maps, command line, and more

```
cat /proc/cpuinfo | grep "model name" | head -1
cat /proc/meminfo | head -5
cat /proc/loadavg
cat /proc/1234/status | grep VmRSS      # actual resident memory used by PID 1234
ls -l /proc/1234/fd                   # every open file descriptor for that process
cat /proc/1234/cmdline                 # the exact command line it was launched with
```

19.3 25.3 /sys

/sys exposes kernel/device configuration in a structured, hierarchical way — a more “modern,” organized counterpart to some of /proc's content, often used for hardware/device tuning (e.g., /sys/class/net/eth0/ for a network interface's settings, /sys/fs/cgroup/ for cgroup controls — directly relevant to how Docker enforces container resource limits, Part 26).

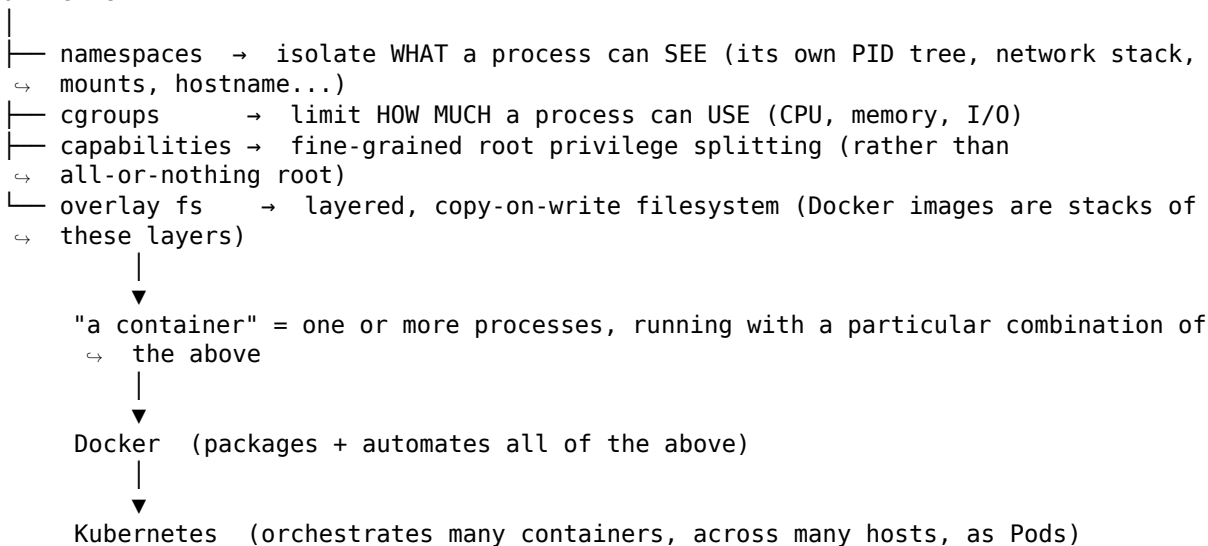
DevOps use case: this is *why* monitoring agents work without any special privileged kernel access — they simply read plain text files under /proc//sys on a regular interval. Understanding this demystifies “how does CloudWatch/Prometheus actually know my CPU usage” — the answer, ultimately, is always “it read /proc.”

20 PART 26 — Linux Containers: The Foundation of Docker & Kubernetes

20.1 26.1 Why This Matters

Docker and Kubernetes do not “virtualize” anything new — they are orchestration layers on top of **kernel features that have existed in Linux for years**: namespaces, cgroups, and (usually) an overlay filesystem. Understanding these makes container behavior — and container *failures* — far less mysterious.

Linux kernel



20.2 26.2 Namespaces

A **namespace** wraps a global kernel resource so that processes inside the namespace see their *own, isolated* view of it, while processes outside see the normal, unmodified global resource.

Namespace	Isolates
PID	Process IDs — a container’s process can be PID 1 <i>inside the container</i> while being just another PID on the host
NET	Network stack — its own interfaces, IP addresses, routing table, port space
MNT	Mount points — its own view of the filesystem
UTS	Hostname and domain name
IPC	Inter-process communication resources (shared memory, semaphores)
USER	UID/GID mapping — a process can be “root” inside the namespace while mapped to an unprivileged UID on the host

```
ls -l /proc/1234/ns/           # see the namespaces a given process belongs to
```

This is exactly why, inside a container, `ps aux` shows only that container’s processes (its own PID namespace), and `hostname` shows the container ID rather than the host’s real hostname.

20.3 26.3 cgroups (Control Groups)

cgroups enforce and account for **resource limits** — CPU shares, memory limits, I/O bandwidth, number of processes — for a group of processes. This is the mechanism behind Docker's `--memory/--cpus` flags and Kubernetes' `resources.limits/resources.requests`.

```
# Inspecting a container's cgroup limits directly (modern cgroup v2 layout)
cat /sys/fs/cgroup/memory.max
cat /sys/fs/cgroup/cpu.max
```

When a container exceeds its memory cgroup limit, the kernel's cgroup-scoped OOM killer terminates a process inside that cgroup — this is exactly the mechanism behind an `OOMKilled` container status in Docker/Kubernetes (same underlying kernel logic as Part 12's OOM killer, just scoped).

20.4 26.4 Capabilities

Traditionally, a process is either fully unprivileged or full **root** (all-powerful). **Capabilities** split root's power into ~40 fine-grained permissions (e.g., `CAP_NET_BIND_SERVICE` to bind privileged ports below 1024, `CAP_SYS_ADMIN` for broad system administration actions) — letting a container run with *just* the specific privileges it actually needs, instead of full root, greatly reducing the blast radius of a compromise.

20.5 26.5 Overlay Filesystems

Docker images are built from **layers** — each Dockerfile instruction typically creates a new read-only layer. At runtime, the container gets a thin **writable layer** on top, and **OverlayFS** (a union filesystem, `overlay2` is Docker's default storage driver) merges all these layers into a single coherent view. This is why images share layers efficiently on disk and why container writes don't affect the underlying image.

20.6 26.6 The Chain, Explicitly

```
Linux namespaces + cgroups + capabilities + overlay fs
    |
    v
"a container" (an isolated, resource-limited process group)
    |
    v
Docker — packages an image (layered filesystem) + runs containers using the above
    ↪ primitives
    |
    v
Kubernetes — schedules and orchestrates many containers (grouped into Pods) across
    ↪ a cluster of nodes,
        using the exact same underlying kernel mechanisms on each node
```

Common mistakes: thinking of a container as “a lightweight VM” — it is not virtualized hardware at all; it's a normal Linux process, isolated by namespaces and limited by cgroups, sharing the **exact same kernel** as the host. This is *why* you can't run a Windows container on a Linux host without a real VM underneath, and why a kernel-level vulnerability can potentially be exploited to escape container isolation (motivating capabilities restrictions, seccomp profiles, and sometimes gVisor/Kata Containers for stronger isolation).

21 PART 27 — Linux on AWS (EC2)

21.1 27.1 EC2 Linux Basics

An EC2 instance is a Linux VM running on AWS's Nitro hypervisor. Everything from Parts 1–26 applies directly — the difference is the surrounding AWS-managed infrastructure: networking (VPC, Security Groups), storage (EBS), and metadata/bootstrapping (user-data, IMDS).

21.2 27.2 SSH and IAM

- Access is typically via SSH key pair (Part 15) or, increasingly, **SSM Session Manager** (no open SSH port needed, access controlled entirely by IAM policy, fully logged).
- An **IAM role** attached to an instance grants it AWS API permissions (e.g., to read from S3, write CloudWatch metrics) **without** embedding static AWS credentials anywhere on the box — the instance fetches temporary, automatically-rotated credentials from the instance metadata service.

21.3 27.3 Instance Metadata Service (IMDS) and IMDSv2

Every EC2 instance can query metadata about itself (instance ID, IAM role credentials, region, user-data) via a special, non-routable link-local address:

```
# IMDSv2 (token-based, current best practice – required, not optional, on modern
↳ instances)
TOKEN=$(curl -s -X PUT "http://169.254.169.254/latest/api/token" \
-H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN"
↳ http://169.254.169.254/latest/meta-data/instance-id
```

IMDSv2 requires this session-token handshake (vs the older IMDSv1, which allowed a plain GET) specifically to mitigate SSRF-based credential theft — a vulnerability in a web app that can be tricked into making an arbitrary GET request could previously steal the IAM role's temporary credentials directly from IMDSv1; IMDSv2's token requirement (using a PUT and a custom header) makes that dramatically harder to trigger via a simple SSRF bug.

21.4 27.4 EBS, Security Groups, cloud-init, and Logs

- **EBS** (Elastic Block Store): network-attached block storage volumes — appear to the instance as normal block devices (`/dev/xvdf`, `/dev/nvme1n1`), managed with the exact same `lsblk/mount/fstab` tooling from Part 10.
- **Security Groups**: stateful, instance-level virtual firewalls (Part 16.3).
- **cloud-init**: the industry-standard tool that consumes **user-data** (a script or cloud-config YAML you supply at launch) to bootstrap a fresh instance on first boot — installing packages, writing config files, creating users, running arbitrary setup commands.

```
#cloud-config
packages:
- nginx
- docker.io
runcmd:
- systemctl enable --now docker
- systemctl enable --now nginx
```

- **CloudWatch Agent**: an installable daemon (itself just a systemd service) that reads `/proc`, disk, and custom log files, and ships metrics/logs to AWS CloudWatch.

- **AMI** (Amazon Machine Image): a snapshot template (root filesystem + boot info) used to launch new instances — essentially “a saved, bootable copy of a configured Linux system.”

21.5 27.5 Troubleshooting a Production EC2 Instance

1. **Instance status checks fail** → check the AWS Console’s system log/console screenshot for boot-time errors (kernel panic, fsck failure, GRUB issue — Part 2.2).
2. **Can’t SSH** → check Security Group inbound rules (port 22 from your IP?), NACL, instance’s own `ufw/iptables` state, whether `sshd` is actually running (via SSM or console output), and key/permission correctness (Part 15).
3. **Instance is unreachable but “running”** → check `cloud-init` logs (`/var/log/cloud-init-output.log`) for a bootstrap failure that left the instance half-configured.
4. **Application not responding** → standard chain from Part 14.9: DNS → Security Group → instance firewall → is the process actually listening (`ss -tulnp`) → application logs/journalctl.
5. **Disk full** → `df -h`, then Part 11’s investigation — remember EBS volumes can be resized live, but the **filesystem** must then also be grown (`growpart` + `resize2fs/xfs_growfs`) to actually use the new space.

Common mistakes: resizing an EBS volume in the AWS console and expecting the OS to “just see” the extra space immediately — the partition and filesystem must also be extended from inside the instance; forgetting IMDSv2’s token requirement when writing custom scripts against instance metadata (a very common migration gotcha when AWS started enforcing IMDSv2-only on new instances).

22 PART 28 — Linux with Docker

22.1 28.1 The Docker Daemon and Container Processes

dockerd (the Docker daemon) is itself a normal Linux process/systemd service. When you run `docker run`, the daemon:

1. Sets up new namespaces (PID, NET, MNT, UTS, IPC) for the container's process.
2. Configures a cgroup to enforce any `--memory/--cpus` limits.
3. Sets up networking (a veth pair connecting the container's network namespace to a bridge on the host, by default).
4. Mounts the image's OverlayFS layers plus a fresh writable layer.
5. Executes your container's entrypoint process **as PID 1 inside its own PID namespace** (but a normal PID on the host — visible via `ps aux` on the host too).

```
docker inspect <container> --format '{{.State.Pid}}' # the container's PID as seen
↳ from the HOST
ps aux | grep <that PID> # confirms it's just a normal
↳ host process
```

22.2 28.2 Volumes and Networking

- **Volumes:** bind-mount a host directory (or a Docker-managed volume) into the container's mount namespace — data persists independently of the container's writable layer, which is destroyed when the container is removed.
- **Networking:** by default, Docker creates a bridge network (`docker0`) — each container gets a veth interface pair, one end in the container's network namespace, the other attached to the bridge on the host, with NAT (Part 14.7) applied for outbound traffic and port-forwarding rules for published ports (`-p 8080:80`).

22.3 28.3 Logs and Resource Limits

```
docker logs -f <container> # container's stdout/stderr
↳ (captured by Docker's logging driver)
docker stats # live cgroup-derived CPU/memory
↳ usage per container
docker run --memory=512m --cpus=1.5 myimage # sets the underlying cgroup
↳ limits directly
```

A container hitting its `--memory` limit is OOM-killed by the cgroup memory controller (Part 26.3) — `docker inspect` will show `OOMKilled: true`.

How Linux concepts map to Docker concepts:

Linux concept	Docker concept
Namespaces	Container isolation (its own PID/network/mount view)
cgroups	<code>--memory</code> , <code>--cpus</code> resource limits
OverlayFS	Image layers + container writable layer
Process (fork/exec)	Container's main process
<code>/var/log</code>	<code>docker logs</code> (via logging drivers)
Bridge networking + NAT	Docker's default bridge network + port publishing

Common mistakes: not setting `--memory` limits at all, letting one container consume unbounded host memory and destabilize every other container on the same host (Part 12's “noisy

neighbor” problem, container edition); assuming container filesystem changes persist — they don’t, unless written to a mounted volume.

23 PART 29 — Linux with Kubernetes

23.1 29.1 The Linux Foundations Kubernetes Depends On

Kubernetes doesn't reinvent anything from Parts 1–28 — it **orchestrates** the exact same Linux primitives (namespaces, cgroups, networking, filesystems) across a fleet of nodes, adding scheduling, self-healing, and service discovery on top.

Kubernetes concept	Underlying Linux mechanism
Pod	One or more containers sharing the same network and IPC namespace (but usually separate mount/PID namespaces per container)
Container resource requests/limits	cgroups (CPU shares/quota, memory limits)
Pod networking (its own IP)	A network namespace + veth pair, same as plain Docker, wired up by the node's CNI plugin
kubelet	A systemd-managed daemon (service unit) running on every node
Container runtime (containerd/CRI-O)	Manages namespaces/cgroups/overlay filesystem, same mechanisms as Part 26–28
Persistent Volumes	Ultimately a mount (local disk, NFS, or cloud block storage like EBS) into the container's mount namespace
kubectl logs	Reads the same container log output Docker/containerd captures (Part 28.3)

23.2 29.2 What Actually Happens When a Pod Runs on a Node

1. kube-scheduler picks a node for the Pod (based on requested cgroup resources vs
↳ available capacity)
2. kubelet (a systemd service on that node) is told to run the Pod
3. kubelet calls the container runtime (containerd) via the CRI (Container Runtime
↳ Interface)
4. containerd pulls the image (if not cached), unpacks it via OverlayFS (Part 26.5)
5. containerd creates namespaces + cgroups for each container in the Pod (Part 26.2,
↳ 26.3)
6. All containers in the Pod SHARE one network namespace — this is why containers in the same Pod can reach each other via localhost, but need Services to reach other
↳ Pods
7. The CNI plugin wires up the Pod's network namespace to the cluster network (assigns
↳ a Pod IP, sets up routes)
8. kubelet continuously monitors the container's process/cgroup, restarting it per the
↳ Pod's restartPolicy
if it exits, and running configured liveness/readiness probes (simple HTTP/exec
↳ health checks, Part 22 territory)

23.3 29.3 Signals in Kubernetes (tying back to Part 7.6)

On Pod termination, kubelet sends **SIGTERM** to the container's main process (PID 1 inside the container), waits up to terminationGracePeriodSeconds (default 30s), then sends **SIGKILL** if it hasn't exited. This is exactly Part 7.6's signal handling — your application code must catch SIGTERM and shut down gracefully, or every rolling deployment will forcibly kill in-flight requests.

Common mistakes: running a process as PID 1 inside a container that doesn't correctly forward/handle signals (a common issue with shell-script entrypoints that don't exec the final process, leaving the shell — not your app — as PID 1, silently swallowing SIGTERM); not understanding that containers *within the same Pod* share a network namespace (so binding to the same port in two containers in one Pod will conflict, exactly as it would for two processes on the same plain Linux host). # PART 30 — DevOps Troubleshooting Scenarios

Each scenario follows: **Symptoms** → **Possible Causes** → **Commands** → **Interpretation** → **Root Cause** → **Fix** → **Prevention**.

23.4 30.1 Server Is Slow

- **Symptoms:** high response latency, sluggish SSH.
- **Possible causes:** CPU saturation, memory pressure/swapping, disk I/O bottleneck, network latency, a runaway process.
- **Commands:** `top`, `vmstat 1`, `free -h`, `iostat -x 1`.
- **Interpretation:** work the Part 22 framework (CPU → Memory → Disk → Network → App) in order.
- **Root cause (example):** a background cron job doing heavy disk I/O overlapped with peak traffic.
- **Fix:** reschedule the job to off-peak, or add I/O throttling/nice/ionice.
- **Prevention:** capacity planning, alerting on load average relative to core count, staggered cron scheduling.

23.5 30.2 CPU Is 100%

- **Symptoms:** `top` shows sustained 100% CPU, requests time out.
- **Causes:** infinite loop/bug, traffic spike beyond capacity, an unexpectedly expensive query/regex, cryptomining malware (rare but real).
- **Commands:** `top`, `ps aux --sort=-%cpu`, `strace -c -p <PID>`.
- **Interpretation:** identify the specific process; if unfamiliar/unexpected, treat as a possible security incident.
- **Root cause (example):** a regex with catastrophic backtracking on malicious input.
- **Fix:** kill/restart the process, patch the regex/code, redeploy.
- **Prevention:** load testing, input validation, autoscaling, resource limits (cgroups/container limits).

23.6 30.3 Memory Is Full

- **Symptoms:** OOM kills in `dmesg`, service crash-looping.
- **Commands:** `free -h`, `dmesg -T | grep -i "killed process"`, `ps aux --sort=-%mem`.
- **Root cause (example):** a memory leak in a long-running app.
- **Fix:** restart the service short-term; fix the leak (heap profiling) long-term.
- **Prevention:** set memory limits (systemd `MemoryMax=`, container `--memory`), add memory alerting well before 100%.

23.7 30.4 Disk Is Full

- **Symptoms:** "No space left on device," services failing to write logs/temp files.
- **Commands:** `df -h`, `du -sh /var/log/* | sort -rh`, `lsuf +L1` (Part 11.3).
- **Root cause (example):** unrotated logs from a chatty service.
- **Fix:** truncate (`> file.log`) or delete safely, configure `logrotate`.
- **Prevention:** `logrotate` on every service by default, disk usage alerting at 80%.

23.8 30.5 SSH Isn't Working

- **Symptoms:** connection refused/timeout/permission denied.
- **Commands:** `ss -tulnp | grep :22` (server, via console/SSM), Security Group/NACL check, `ssh -v` (verbose client output), check key permissions (`chmod 600`).
- **Root cause (example):** Security Group inbound rule doesn't allow your current IP.
- **Fix:** update the Security Group rule.
- **Prevention:** use SSM Session Manager where possible to remove SSH-port exposure entirely.

23.9 30.6 Website Isn't Accessible

- **Symptoms:** browser can't load the site.
- **Commands:** work the full chain from Part 14.9 — `dig`, `ping`, `tracert`, `curl -v`, `ss -tulnp` on the server.
- **Root cause (example):** DNS record points to an old/decommissioned IP.
- **Fix:** correct the DNS record; wait for TTL/propagation.
- **Prevention:** DNS change checklists, lower TTLs before planned cutovers.

23.10 30.7 Port Isn't Listening

- **Symptoms:** connection refused on a specific port.
- **Commands:** `ss -tulnp | grep <port>` on the server itself.
- **Root cause (example):** the app crashed on startup and never actually bound the port.
- **Fix:** check app/service logs (`journalctl -u <svc>`) for the startup error.
- **Prevention:** startup health checks, `Restart=on-failure` in `systemd`, readiness probes in Kubernetes.

23.11 30.8 DNS Isn't Resolving

- **Symptoms:** `curl` fails with "could not resolve host."
- **Commands:** `dig example.com`, `cat /etc/resolv.conf`, `dig @8.8.8.8 example.com` (bypass local resolver to isolate the issue).
- **Root cause (example):** internal DNS resolver (e.g., VPC resolver) unreachable due to a Security Group change.
- **Fix:** restore resolver reachability.
- **Prevention:** monitor DNS resolution as part of health checks, not just HTTP status.

23.12 30.9 Service Won't Start

- **Symptoms:** `systemctl start` fails or the service immediately exits.
- **Commands:** `systemctl status <svc>`, `journalctl -u <svc> -n 100`.
- **Root cause (example):** a typo in a config file the service fails to parse on startup.
- **Fix:** correct the config, `daemon-reload` if the unit file itself changed, restart.
- **Prevention:** config validation in CI before deploy (many services support a `--check-config` style flag).

23.13 30.10 Service Keeps Restarting (Crash Loop)

- **Symptoms:** `systemctl status` shows repeated activating/failed cycling, or Kubernetes shows `CrashLoopBackOff`.
- **Commands:** `journalctl -u <svc> -f` (watch the actual crash reason), `kubectl logs --previous` (K8s equivalent).

- **Root cause (example):** missing required environment variable causing the app to exit on startup.
- **Fix:** supply the correct configuration.
- **Prevention:** startup config validation with clear error messages, staging environment parity testing.

23.14 30.11 Nginx Returns 502 (Bad Gateway)

- **Symptoms:** Nginx itself responds, but with 502.
- **Meaning:** Nginx (as a reverse proxy) could not get a valid response from the **upstream** application server — often because the upstream is down, crashed, or refusing the connection.
- **Commands:** `ss -tulnp | grep <upstream-port>`, check the upstream app's own logs, `curl` the upstream directly (bypassing Nginx) to isolate.
- **Fix:** restart/fix the upstream app.
- **Prevention:** health checks on upstreams, graceful restarts, connection draining during deploys.

23.15 30.12 Nginx Returns 504 (Gateway Timeout)

- **Symptoms:** Nginx waits, then times out.
- **Meaning:** the upstream **is** reachable but took too long to respond.
- **Commands:** check upstream app response time/logs, database slow-query logs, `io-stat/top` on the upstream host.
- **Root cause (example):** a slow, unindexed database query under load.
- **Fix:** optimize the query/add an index; consider raising Nginx's `proxy_read_timeout` only as a stopgap, not a real fix.
- **Prevention:** query performance monitoring, load testing, timeout budgets defined end-to-end.

23.16 30.13 Permission Denied

- **Symptoms:** a command/app fails with “Permission denied.”
- **Commands:** `ls -l` the target, `id` the user the process runs as, `stat` for detailed meta-data, check SELinux/AppArmor denials (Part 21.3).
- **Root cause (example):** a deploy script changed a file's owner to root, but the app runs as a non-root service user.
- **Fix:** `chown/chmod` correctly for the service's actual runtime user.
- **Prevention:** deploy scripts should always explicitly set ownership/permissions as a final, verified step.

23.17 30.14 Process Is Stuck

- **Symptoms:** a process appears “hung,” not responding, no CPU usage.
- **Commands:** `ps -o stat,pid,cmd -p <PID>` (check for D state — Part 7.4), `strace -p <PID>` to see what it's currently blocked on.
- **Root cause (example):** blocked on a slow/hung disk or network filesystem (NFS) call — D state, uninterruptible.
- **Fix:** address the underlying storage/network issue; a D-state process usually cannot even be killed with `SIGKILL` until the I/O completes or times out.
- **Prevention:** avoid unreliable network filesystems for critical paths; set I/O timeouts.

23.18 30.15 Zombie Process

- See Part 7.3 in full. **Fix:** the *parent* process must be fixed or restarted — you cannot kill a zombie directly, it's already dead and just awaiting reaping.

23.19 30.16 High Load Average

- See Part 13.2. Always compare against `nproc` before concluding anything is actually wrong.

23.20 30.17 Network Latency

- **Symptoms:** slow requests, high round-trip time.
- **Commands:** `ping`, `mtr/traceroute` (identify which hop introduces delay), `ss -i` (check for retransmits on existing connections).
- **Root cause (example):** cross-region traffic taking a suboptimal path, or an overloaded intermediate load balancer.
- **Fix:** route traffic through a closer region/CDN edge, scale the bottlenecked component.
- **Prevention:** geo-aware architecture, CDN usage, latency-based routing/monitoring.

23.21 30.18 Packet Loss

- **Symptoms:** `ping` shows dropped packets, intermittent connection failures.
- **Commands:** sustained `ping -c 100`, `mtr` to localize the hop, `tcpdump` for deeper inspection.
- **Root cause (example):** a saturated network link or a failing piece of network hardware upstream.
- **Fix:** often outside your direct control (upstream provider) — escalate with the exact `mtr` evidence; if it's your own instance, check for bandwidth saturation.
- **Prevention:** network monitoring/alerting on loss percentage, redundant paths where critical.

23.22 30.19 Container Cannot Connect to Another Container

- **Symptoms:** connection refused/timeout between two containers.
- **Commands:** verify both are on the **same** Docker network (`docker network inspect`), verify the target's actual listening port from inside its own namespace, check container-level firewall/security policies (Kubernetes NetworkPolicy, if applicable).
- **Root cause (example):** the two containers are on different, non-connected Docker networks/K8s namespaces without a Service/DNS entry linking them.
- **Fix:** attach both to the same network, or create the correct Kubernetes Service.
- **Prevention:** standardized network/service naming conventions, documented service discovery patterns.

23.23 30.20 Application Crashes

- **Symptoms:** process exits unexpectedly.
- **Commands:** `journalctl -u <svc>` or `docker logs/kubectl logs --previous` for the exit reason/stack trace, `dmesg` to rule out an OOM kill masquerading as an app crash.
- **Root cause (example):** an unhandled exception on a specific edge-case input.
- **Fix:** patch the bug, add proper error handling.
- **Prevention:** better test coverage, structured error handling, crash reporting/alerting.

23.24 30.21 Log Files Fill the Disk

- See Part 11.4 in full — logrotate, fixing crash loops, correcting log verbosity.

23.25 30.22 EC2 Instance Cannot Be Accessed

- See Part 27.5's full troubleshooting checklist — instance status checks, Security Group/NACL, cloud-init logs, SSM as a fallback access path when SSH itself is the problem.

23.26 30.23 Docker Container Keeps Restarting

- **Symptoms:** `docker ps` shows repeated restarts, or Restarting status.
- **Commands:** `docker logs <container>` (the actual crash reason), `docker inspect <container>` (check OOMKilled, ExitCode, restart policy).
- **Root cause (example):** container exceeding its --memory limit under real load, getting OOM-killed by its cgroup repeatedly.
- **Fix:** raise the memory limit appropriately (based on actual measured usage) or fix a leak in the app.
- **Prevention:** load-test containers with realistic traffic before setting limits, monitor `docker stats/cgroup` memory usage trends over time. # PART 31 — Master Command Cheat Sheet

Organized by category. Format: **COMMAND → PURPOSE → COMMON OPTIONS → EXAMPLE → DEVOPS USE CASE.**

23.27 Navigation

- **pwd** → print working dir → — → `pwd` → confirm script's actual run location
- **ls** → list contents → `-l -a -h -t -R` → `ls -lath` → inspect log dirs by recency/size
- **cd** → change dir → `-, ~` → `cd -` → jump back after a detour
- **find** → search by criteria → `-name -type -mtime -size -exec` → `find /var/log -mtime +7 -delete` → log cleanup [!]
- **tree** → visual dir tree → `-L n` → `tree -L 2` → understand unfamiliar repo/app layout

23.28 Files

- **touch** → create/update timestamp → — → `touch app.log`
- **mkdir** → create dir → `-p` → `mkdir -p /opt/app/releases`
- **cp** → copy → `-r -p -v` → `cp -rp src/ backup/`
- **mv** → move/rename → — → `mv old.log old.log.bak`
- **rm** → delete → `-r -f -i` → `[!] rm -rf dir/`
- **ln** → link → `-s` → `ln -s target link`
- **file** → identify type → — → `file binary`
- **stat** → metadata → — → `stat file`

23.29 Text Processing

- **cat/tac** → print (forward/reverse) → — → `cat file.txt`
- **less/more** → paginate → `+F` → `less +F app.log`
- **head/tail** → first/last N lines → `-n, -f` → `tail -f app.log`
- **grep/egrep** → search → `-r -i -n -v -E -A -B -C` → `grep -ri error app.log`
- **sed** → stream edit → `-i` → `sed -i 's/old/new/g' file`

- **awk** → field processing → -F → `awk -F, '{print $1}' file.csv`
- **cut** → extract fields → -d -f → `cut -d: -f1 /etc/passwd`
- **sort/uniq** → sort/dedupe → -n -r -k, -c → `sort file \|| uniq -c`
- **wc** → count → -l -w -c → `wc -l file`
- **xargs** → build commands from stdin → — → `find . -name "*.tmp" \|| xargs rm`
- **tee** → split output to file+stdout → -a → `cmd \|| tee out.log`

23.30 Permissions

- **chmod** → change perms → symbolic/numeric → `chmod 755 script.sh`
- **chown** → change owner → -R → `chown -R app:app /opt/app`
- **chgrp** → change group → — → `chgrp devops file`
- **umask** → default perms → — → `umask 022`
- **sudo/su** → privilege escalation → — → `sudo systemctl restart nginx`
- **getfacl/setfacl** → ACLs → -m -x → `setfacl -m u:bot:rw file`

23.31 Users

- **useradd/adduser** → create user → -m -s → `useradd -m -s /bin/bash deploy`
- **usermod** → modify user → -aG → `usermod -aG docker $USER`
- **userdel** → delete user → -r → `userdel -r deploy`
- **passwd** → set password → — → `passwd deploy`
- **groupadd/groupdel** → manage groups → — → `groupadd devops`
- **id/groups/whoami** → identity info → — → `id aryan`
- **who/w/last** → session/login info → — → `last -n 10`

23.32 Processes

- **ps** → snapshot → aux, -ef → `ps aux --sort=-%cpu`
- **top/htop** → live view → — → `top`
- **pstree** → process tree → -p → `pstree -p`
- **kill/killall/pkill** → send signal → -9 -HUP -TERM → `kill -TERM 1234`
- **nohup** → survive hangup → & → `nohup ./run.sh &`
- **nice/renice** → set priority → -n → `nice -n 10 ./job.sh`
- **jobs/bg/fg** → job control → — → `fg %1`

23.33 Services (systemd)

- **systemctl start/stop/restart/reload** → manage service state → — → `systemctl restart nginx`
- **systemctl enable/disable** → manage boot-time autostart → --now → `systemctl enable --now myapp`
- **systemctl status** → check state/recent logs → — → `systemctl status myapp`
- **systemctl daemon-reload** → reload unit file cache → — → after editing a .service file
- **journalctl** → query logs → -u -f -p --since → `journalctl -u myapp -f`

23.34 Networking

- **ip addr/link/route** → interfaces/routing → — → `ip addr show`
- **ping/traceroute** → reachability/path → -c → `ping -c 4 8.8.8.8`
- **ss/netstat** → sockets/ports → -tulnp → `ss -tulnp`
- **dig/nslookup/host** → DNS → +short → `dig example.com +short`

- **curl/wget** → HTTP requests/downloads → `-I -v -o → curl -I https://example.com`
- **nc** → raw TCP/UDP → `-zv → nc -zv host 443`
- **tcpdump** → packet capture → `-i -port → tcpdump -i eth0 port 443`

23.35 Storage

- **lsblk/blkid** → list block devices → `-f → lsblk -f`
- **df/du** → space usage (fs/dir) → `-h -i -s → df -h, du -sh *`
- **mount/umount** → attach/detach fs → `— → mount \ | grep xvdf`
- **fdisk/parted** → partition → `[!] → parted /dev/xvdf`
- **mkfs** → format ([!] destructive) → `.ext4 .xfs → [!] mkfs.ext4 /dev/xvdf1`
- **fsck** → check/repair fs → `— → fsck /dev/xvdf1`

23.36 Monitoring

- **free** → memory summary → `-h → free -h`
- **vmstat** → memory/cpu/io trend → `l → vmstat l 5`
- **iostat** → per-device disk I/O → `-x l → iostat -x l`
- **sar** → historical stats → `-r → sar -r`
- **uptime/lscpu/nproc** → CPU/load info → `— → uptime`
- **lsof** → open files/sockets → `-i :port → lsof -i :8080`

23.37 Packages

- **apt update/upgrade/install** (Debian/Ubuntu) → package mgmt → `-y → apt install -y nginx`
- **dnf/yum install/update** (RHEL family) → package mgmt → `-y → dnf install -y nginx`
- **dpkg -l / rpm -qa** → list installed packages → `— →`

23.38 SSH

- **ssh** → remote shell → `-i -L -R -J → ssh -i key.pem user@host`
- **scp/sftp** → file transfer → `— → scp file user@host:/path`
- **ssh-keygen** → generate keys → `-t → ssh-keygen -t ed25519`

23.39 Security

- **ufw** → firewall (Ubuntu) → `allow deny enable → ufw allow 22/tcp`
- **fail2ban-client** → check/manage bans → `status → fail2ban-client status sshd`
- **getenforce/aa-status** → MAC status → `— → check SELinux/AppArmor mode`

23.40 Archives & Compression

- **tar** → archive → `-czvf -xzvf → tar -czvf backup.tar.gz /opt/app`
- **gzip/gunzip** → compress/decompress → `-k keep original → gzip -k file`
- **zip/unzip** → zip format → `— → unzip archive.zip`

```
tar -czvf backup-$(date +%F).tar.gz /opt/app           # create compressed archive
tar -xzvf backup-2026-08-11.tar.gz -C /restore/path    # extract to a target dir
tar -tzvf backup.tar.gz                                # list contents WITHOUT
↪ extracting (safe check first)
```

23.41 Scheduling

- **crontab -e/-l** → manage recurring jobs → — → **crontab -e**
- **at** → one-time scheduled job → — → **at 22:00**
- **systemctl list-timers** → view systemd timers → — →

23.42 Debugging

- **strace/ltrace** → syscall/library call tracing → -p -c -f → **strace -p 1234**
- **dmesg** → kernel messages → -T → **dmesg -T \|| tail**
- **lsof** → open files → +L1 → find deleted-but-open files

23.43 System Information

- **uname -a** → kernel/OS info → — →
- **hostnamectl** → hostname + OS summary → — →
- **lscpu/lsblk/lsusb/lspci** → hardware inventory → — →
- **cat /etc/os-release** → distro identification → — → # PART 32 — Linux DevOps Interview Preparation

Each entry: **short answer** you'd give out loud, then a **detailed explanation**, then a **real-world example**.

23.44 Linux Fundamentals

Q: What's the difference between the Linux kernel and a Linux distribution? Short answer: the kernel is one program managing hardware/processes; a distro bundles the kernel with userland tools, a package manager, and defaults. Detail: Ubuntu, RHEL, and Amazon Linux all typically run very similar/same-family kernels but differ in package management, release cadence, and default configuration. Example: "Ubuntu uses APT and a 2-year LTS cycle, Amazon Linux uses DNF and is AWS-optimized, but both are built on the Linux kernel."

Q: Explain User → Shell → Kernel → Hardware. Short answer: the shell parses your command, the resulting program makes system calls, the kernel executes them against hardware/state. Detail: this is the core mental model for reasoning about "where in the stack" a problem lives. Example: **cat file.txt** → bash execs **cat** → **cat** calls **open()/read()** syscalls → kernel reads from disk → data returned to **cat** → printed to terminal.

23.45 Filesystem

Q: What is an inode, and what's the difference between a hard link and a symlink? Short answer: an inode holds a file's metadata and data-block pointers, separate from its name; a hard link is a second name for the same inode, a symlink is a separate file pointing to a path. Detail: hard links can't cross filesystems or link directories; symlinks can cross filesystems and dangle if the target moves. Example: "Deployment tooling often symlinks current -> releases/v1.4.2 for atomic, zero-downtime cutovers."

Q: Why doesn't disk space free up immediately after rm-ing a large file? Short answer: if a running process still has the file open, the kernel keeps its data blocks allocated until the last file descriptor closes. Detail: **df** shows it as used, **du/ls** show it as gone — check with **lsof +L1**. Example: "A log file kept growing until disk-full even after **rm**, because the app still held the deleted file open — restarting the app released the space."

23.46 Permissions

Q: Explain chmod 755 vs chmod 644. Short answer: 755 = owner rwx, group/other r-x (typical for scripts/executables); 644 = owner rw, group/other read-only (typical for regular config/data files). Detail: the three octal digits map to user/group/other, each summing $r=4+w=2+x=1$. Example: “A deploy script needs 755 so the service user can execute it; its config file should usually be 644 or tighter (600 if it holds secrets).”

Q: What’s the difference between SUID and sticky bit? Short answer: SUID makes an executable run with the file owner’s privileges; sticky bit on a directory restricts deletion to the file’s own owner. Detail: `/usr/bin/passwd` is SUID root so any user can update `/etc/shadow` through it safely; `/tmp` has the sticky bit so users can’t delete each other’s files in a shared world-writable directory. Example: `ls -ld /tmp` shows `drwxrwxrwt` — that trailing `t` is the sticky bit.

23.47 Processes

Q: What’s a zombie process, and how do you fix it? Short answer: a finished process whose exit status hasn’t been collected by its parent yet; you fix it by fixing/restarting the **parent**, not the zombie itself. Detail: it shows as `<defunct>` in `ps`, consumes almost no resources alone, but many can exhaust the process table. Example: “A buggy supervisor script forking workers without ever calling `wait()` accumulated thousands of zombies over weeks.”

Q: SIGTERM vs SIGKILL — when would you use which? Short answer: SIGTERM asks a process to shut down gracefully (can be caught); SIGKILL force-terminates immediately, no cleanup, cannot be caught. Detail: always try SIGTERM first; SIGKILL is a last resort since it can leave data half-written or resources uncleaned. Example: “Kubernetes sends SIGTERM on pod termination and only escalates to SIGKILL after the grace period — apps must handle SIGTERM to avoid dropping in-flight requests.”

23.48 Networking

Q: Walk me through troubleshooting ‘I can’t reach my web server.’ Short answer: work the chain — DNS → IP reachability → routing → firewall (SG/NACL/host firewall) → is the port listening → is the app actually healthy. Detail: use `dig`, `ping/traceroute`, then `ss -tulpn` on the server, then `curl -v`. Example: “Turned out the Security Group allowed port 80 but not 443, so HTTPS specifically failed while HTTP worked.”

Q: What’s the difference between TCP and UDP? Short answer: TCP is connection-oriented and reliable (handshake, retransmission, ordering); UDP is connectionless and best-effort, lower overhead. Detail: TCP suits HTTP/SSH/databases; UDP suits DNS queries, streaming, some monitoring. Example: “We used UDP for a metrics pipeline where occasional dropped datapoints were acceptable in exchange for lower latency and overhead.”

23.49 SSH

Q: How does SSH key-based authentication work, and why disable password auth? Short answer: the server checks a client’s public key (in `authorized_keys`) against a challenge signed by the client’s private key; disabling password auth removes the brute-forceable attack surface entirely. Detail: private keys stay local and must be `chmod 600`; combine with `PermitRootLogin no` and `fail2ban` for defense in depth. Example: “We hardened `sshd_config` to key-only auth plus an `AllowUsers` allowlist after a wave of brute-force attempts appeared in `/var/log/auth.log`.”

23.50 systemd

Q: What does `systemctl enable` do vs `systemctl start`? Short answer: `enable` configures a service to auto-start at boot (doesn't start it now); `start` runs it now (doesn't persist across reboot). Use `--now` for both. Detail: this trips up a lot of engineers who start a service, reboot the box, and are confused it's not running. Example: `systemctl enable --now myapp`.

Q: How would you debug a service that fails to start? Short answer: `systemctl status` for the immediate reason/exit code, then `journalctl -u <svc> -n 100` for full context. Detail: common causes are config parse errors, missing env vars, permission issues, or a port already in use. Example: "journalctl showed the app exiting because a required `DATABASE_URL` env var wasn't set in the unit file."

23.51 Storage

Q: `df` shows the disk full but `du` doesn't find large files. Why? Short answer: likely a deleted-but-still-open file, or inode exhaustion rather than actual space exhaustion. Detail: check `lsof +L1` for held-open deleted files, and `df -i` for inode usage separately from `df -h`'s space usage. Example: "Millions of tiny session files exhausted inodes long before disk space ran out — `df -h` looked fine, `df -i` showed 100%."

23.52 Memory

Q: Why does `free -h` show most of my RAM as 'used' even though the system feels fine? Short answer: most of it is usually the kernel's page cache — reclaimable, not truly locked by applications; check the available column. Detail: high cache usage is normal, healthy behavior; only sustained low available plus swap activity indicates real pressure. Example: "New engineers often panic at `free`'s used column until they learn to read available instead."

23.53 CPU

Q: How do you interpret 'load average: 8.0, 6.5, 5.0'? Short answer: meaningless without knowing core count — compare against `nproc`. Detail: load average is the number of processes wanting CPU (running or blocked), not a percentage. Example: "8.0 on a 16-core box is 50% headroom; 8.0 on a 4-core box means the system is significantly overloaded."

23.54 Security

Q: SELinux vs AppArmor — what's the practical difference? Short answer: both are mandatory access control on top of standard permissions; SELinux is label-based and very granular (RHEL family default), AppArmor is path-based and simpler (Ubuntu/Debian default). Detail: both can silently block an otherwise-permitted action — check `ausearch/audit` logs or AppArmor denials before assuming it's a plain permissions bug. Example: "Nginx couldn't proxy to a custom backend port until we adjusted the SELinux `httpd_can_network_connect` boolean."

23.55 Bash

Q: Why should you use `set -euo pipefail` in production scripts? Short answer: `-e` stops on the first failing command, `-u` errors on unset variables, `pipefail` fails the pipeline if any stage fails — together they prevent scripts from silently continuing in a broken state. Detail: without it, a failed `cd` or a typo'd variable can let a script keep running destructively against

the wrong path. Example: “A deploy script without `set -e` continued past a failed build step and deployed a stale artifact.”

23.56 Troubleshooting

Q: Nginx returns 502 vs 504 — what’s the difference? Short answer: 502 means the upstream failed/refused the connection entirely; 504 means the upstream was reachable but didn’t respond in time. Detail: 502 points to “is the app even running/reachable,” 504 points to “the app is slow” (often a database query). Example: “504s correlated exactly with a nightly batch job locking rows in the database, causing app queries to stall.”

23.57 Docker/Linux

Q: What Linux kernel features does Docker actually rely on? Short answer: namespaces (isolation), cgroups (resource limits), and OverlayFS (layered image filesystem). Detail: a container is not a lightweight VM — it’s a normal host process, isolated and limited by these kernel mechanisms, sharing the host kernel. Example: “Because containers share the host kernel, we can’t run a container built for a much newer kernel’s syscalls on an old host kernel.”

23.58 Kubernetes/Linux

Q: Why do containers in the same Pod share localhost but containers in different Pods don’t? Short answer: containers within one Pod share a single network namespace; different Pods each get their own. Detail: this is the same Linux namespace mechanism from Part 26, just applied at the Pod boundary by the container runtime/CNI. Example: “A sidecar container reaches its main app container over `localhost:PORT` because they’re namespace-mates within the same Pod.”

23.59 AWS/Linux

Q: What is IMDSv2, and why does it matter for security? Short answer: a hardened version of EC2’s instance metadata service requiring a session token (via PUT + custom header) instead of a plain GET, mitigating SSRF-based credential theft. Detail: IMDSv1 let a vulnerable app be tricked into leaking IAM role credentials via a simple GET request; IMDSv2 requires an extra step that’s much harder to trigger blindly through SSRF. Example: “We enforced IMDSv2-only on all instances after a security review flagged IMDSv1 as an SSRF risk in one of our public-facing apps.” # PART 33 — Hands-On Labs (50+)

Work through these in order — each builds on skills from earlier ones. Use a real Ubuntu 24.04 VM, WSL2, or EC2 instance.

23.60 Foundations (Labs 1-10)

1. **Explore the filesystem** — `cd` through `/etc`, `/var`, `/proc`, `/sys`; identify 5 files you don’t recognize and `stat` each.
2. **Create users and groups** — create a devops group and two users, add both to the group, verify with `id`.
3. **Configure permissions** — create a shared directory with SGID set so new files inherit the group automatically.
4. **Create symbolic and hard links** — demonstrate the difference; delete the original file and observe what happens to each link.
5. **Find large files** — use `find` to locate every file over 100MB on the system.

6. **Analyze a log file** — download/generate a sample access log; extract the top 10 IPs and top 5 status codes with `awk/sort/uniq`.
7. **Practice redirection** — write a command that separately captures stdout and stderr from a script that produces both.
8. **Build a pipeline** — chain at least 4 commands together to answer a real question about a text file.
9. **Inode investigation** — create 10,000 tiny files in a test directory, then check `df -i` before/after.
10. **Hidden files** — find and inspect every dotfile in your home directory.

23.61 Processes & Services (Labs 11-20)

11. **Create a systemd service** — package a simple script as a managed, auto-restarting service.
12. **Configure SSH keys** — generate a keypair, deploy the public key to a second host/VM, disable password auth.
13. **Configure a firewall** — set up `ufw` with a deny-by-default policy, explicitly allowing only SSH and HTTP.
14. **Schedule a cron job** — write a cron entry that runs a cleanup script every night and logs its output.
15. **Monitor CPU** — write a one-liner that alerts (prints a warning) if load average exceeds core count.
16. **Monitor memory** — write a script that logs `free -h` output every minute to a file.
17. **Diagnose disk usage** — fill a test volume artificially, then use `df/du` to locate and fix it.
18. **Diagnose network connectivity** — use `ping`, `traceroute`, and `ss` to fully map the path to a public site.
19. **Configure Nginx** — install Nginx, serve a static page, and set up a reverse proxy to a simple backend app.
20. **Deploy an application** — deploy a small Python/Node app as a systemd-managed service behind Nginx.

23.62 Automation & Troubleshooting (Labs 21-30)

21. **Create a backup script** — back up a directory to a timestamped `.tar.gz`, rotate to keep only the last 7.
22. **Create a monitoring script** — poll an HTTP health endpoint every 30s and log failures.
23. **Troubleshoot a failed service** — intentionally break a systemd unit's config, then diagnose and fix it using only `systemctl/journalctl`.
24. **Troubleshoot high CPU** — write a small script that spins the CPU, then find and kill it using `top/ps`.
25. **Troubleshoot memory exhaustion** — write a script that allocates memory until OOM-killed, then find the evidence in `dmesg`.
26. **Troubleshoot disk exhaustion** — fill a disk on purpose (in a test VM!) and practice safely recovering space.
27. **Troubleshoot DNS** — misconfigure `/etc/resolv.conf` temporarily and diagnose the resulting failure.
28. **Troubleshoot ports** — start a service on the wrong port and diagnose it with `ss -tulpn`.
29. **Troubleshoot Docker networking** — create two containers on separate networks, observe the failure, then fix it.
30. **Investigate a production-like outage** — combine a partner/mentor-created “broken” VM and diagnose the root cause end-to-end.

23.63 Storage & Filesystem Depth (Labs 31-35)

31. **Attach and mount a new volume** — attach a second disk (VM or EBS), partition, format as ext4, mount, and persist via /etc/fstab.
32. **Resize a filesystem** — grow a volume and extend the filesystem live (growpart + resize2fs).
33. **Configure swap** — create and enable a swap file, verify with free -h and swapon --show.
34. **Set up log rotation** — write a logrotate config for a custom application log directory.
35. **Recover a “deleted but open” file’s space** — reproduce Lab 9’s scenario and fix it using lsof +L1.

23.64 Networking & Security Depth (Labs 36-42)

36. **Set up SSH port forwarding** — use local forwarding to reach a database only exposed on a private host.
37. **Configure a jump host** — SSH into a private-subnet host via a bastion using -J.
38. **Capture and inspect packets** — use tcpdump to capture traffic on port 80/443 and interpret the output.
39. **Harden SSH** — implement the full checklist from Part 15.4 on a test host.
40. **Set up fail2ban** — configure it to ban IPs after 5 failed SSH attempts, then test it (carefully, on a test host).
41. **Audit SUID binaries** — run the find / -perm -4000 audit and research what each result is for.
42. **Compare Security Group vs host firewall** — reproduce a scenario where SG allows a port but ufw blocks it (and vice versa), and diagnose which layer is at fault.

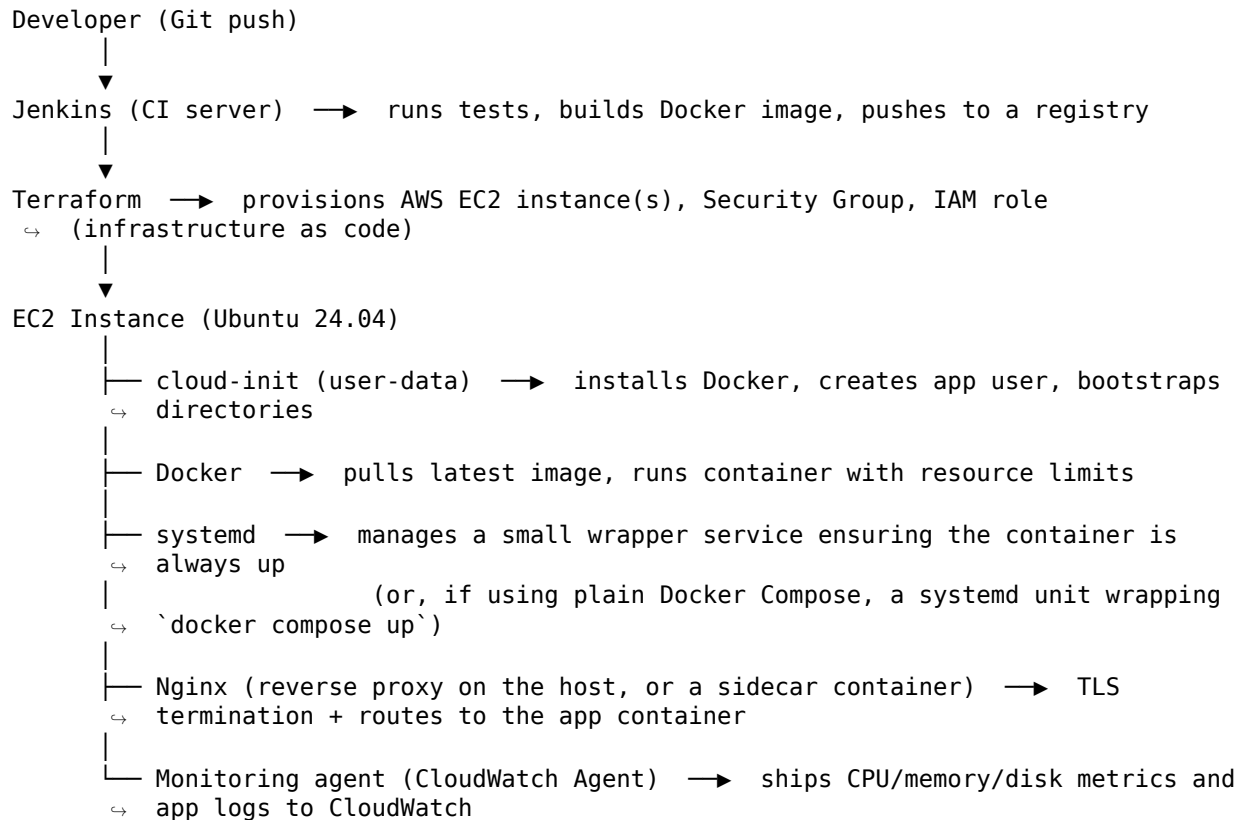
23.65 Containers & Cloud (Labs 43-50+)

43. **Run your first container** — docker run an Nginx container, inspect its namespaces from the host.
44. **Set container resource limits** — run a container with --memory and deliberately trigger an OOM kill.
45. **Build a custom image** — write a Dockerfile for a simple app, build, and run it.
46. **Persist data with a volume** — demonstrate that data survives container removal only when using a volume.
47. **Connect containers via a network** — link two containers by name on a custom Docker network.
48. **Launch an EC2 instance with user-data** — use cloud-init to bootstrap Nginx + Docker on first boot.
49. **Attach an IAM role and use IMDSv2** — retrieve temporary credentials from instance metadata using the token flow.
50. **Investigate an EC2 boot failure** — intentionally break /etc/fstab on a test instance, observe the boot failure via console output, and recover it.
51. **(Capstone-adjacent) Deploy behind a load balancer** — run two app instances/containers and put a load balancer or Nginx upstream block in front, testing failover when one is stopped.
52. **(Capstone-adjacent) Full CI-to-prod dry run** — manually perform the deploy sequence you’ll automate in Part 34’s project: build → push image → SSH/SSM to server → pull → restart service → verify health. # PART 34 — Real DevOps Project: Linux + Git + Bash + Python + Nginx + systemd + Docker + AWS EC2 + Terraform + Jenkins + Monitoring

23.66 Project Goal

Build and deploy a small web application end-to-end, using every layer covered in this course, so the Linux concepts stop being abstract and become the actual mechanics of a real deployment pipeline.

23.67 Architecture



23.68 Step-by-Step

1. Application (Python + Git) - A small Flask/FastAPI app exposing / and /health. - Version-controlled in Git; .gitignore excludes secrets/venvs.

2. Containerize (Docker)

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8080
CMD ["python3", "app.py"]
```

3. Infrastructure as Code (Terraform) - Define a VPC/subnet (or reuse an existing one), a Security Group (22 from your IP only, 80/443 from anywhere), an IAM role (CloudWatch + ECR/registry pull permissions), and an EC2 instance resource referencing a user_data cloud-init script. - terraform plan → review → terraform apply.

4. Bootstrap (cloud-init / user-data, Bash)


```
#cloud-config
packages:
  - docker.io
  - nginx
runcmd:
  - systemctl enable --now docker
  - usermod -aG docker ubuntu
  - mkdir -p /opt/myapp
```

5. CI/CD (Jenkins) - A Jenkinsfile pipeline: checkout → run tests → docker build → push to registry (e.g., ECR) → SSH/SSM into the EC2 instance → docker pull + docker restart (or trigger the systemd wrapper unit to restart).

6. Process supervision (systemd)

```
# /etc/systemd/system/myapp.service
[Unit]
Description=MyApp container wrapper
After=docker.service
Requires=docker.service

[Service]
Restart=always
ExecStartPre=/usr/bin/docker stop myapp
ExecStartPre=/usr/bin/docker rm myapp
ExecStart=/usr/bin/docker run --name myapp --memory=512m --cpus=1 -p 8080:8080
↳ myregistry/myapp:latest
ExecStop=/usr/bin/docker stop myapp

[Install]
WantedBy=multi-user.target
```

7. Reverse proxy (Nginx)

```
server {
    listen 80;
    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
    }
}
```

8. Monitoring - Install the CloudWatch Agent (or Prometheus node_exporter if self-hosting monitoring) to ship CPU/memory/disk metrics and application logs off-box. - Add a health-check script (Part 19.7) as a cron job or systemd timer, alerting on repeated failures.

9. Verify the whole chain

```
curl -I http://<ec2-public-ip>/health
journalctl -u myapp -f
docker logs -f myapp
```

This project deliberately touches every layer of the course: Git (source control) → Bash (bootstrap/scripts) → Docker (namespaces/cgroups/overlayfs from Part 26) → systemd (process supervision from Part 8) → Nginx (reverse proxy from Part 14) → Terraform (infra as code) → Jenkins (automation) → AWS EC2/IAM (Part 27) → CloudWatch (monitoring, reading /proc under the hood, Part 25).

24 Complete Linux Roadmap

1. **Weeks 1-2** — Fundamentals, filesystem, permissions, users/groups, the CLI (Parts 1-6).
 2. **Weeks 3-4** — Processes, systemd, logging, storage, memory, CPU (Parts 7-13).
 3. **Weeks 5-6** — Networking, SSH, firewalls, package management, Bash scripting, cron (Parts 14-20).
 4. **Week 7** — Security and performance (Parts 21-23).
 5. **Week 8** — Kernel internals, containers, Docker, Kubernetes, AWS (Parts 24-29).
 6. **Ongoing** — Troubleshooting scenarios, labs, and the capstone project, revisited continuously as you hit real issues at work.
-

25 30-Day Linux Learning Plan

Days	Focus	Deliverable
1-3	Part 1-3: fundamentals, boot, filesystem	Can explain User→Shell→Kernel and navigate the FHS from memory
4-6	Part 4-5: CLI + permissions	Comfortable with pipelines, redirection, chmod/chown
7-8	Part 6: users & groups	Labs 2-3 done
9-11	Part 7-8: processes + systemd	Labs 11 done; can write and debug a custom .service
12	Part 9: logging	Comfortable with journalctl filters
13-15	Part 10-13: storage, memory, CPU	Labs 17, 31-33 done
16-19	Part 14-16: networking, SSH, firewalls	Labs 12-13, 18, 36-42 done
20	Part 17-18: packages + env vars	—
21-23	Part 19-20: Bash scripting + cron	Health-check script (19.7) written and scheduled
24	Part 21-22: security + performance	Ran the SUID audit, worked through the CPU/memory/disk framework once for real
25	Part 23: advanced tools	Used strace/lsof on a real process at least once
26-27	Part 24-26: kernel + containers	Labs 43-47 done
28	Part 27-29: AWS + Docker + Kubernetes connections	Lab 48-50 done
29	Part 30: troubleshooting scenarios	Re-read all 23 scenarios, could talk through each unaided
30	Part 34: capstone project	At least the Terraform + EC2 + Docker + Nginx portion deployed and reachable

25.1 Daily Practice Schedule (suggested, ~1.5-2 hrs/day)

- 30 min — read/review a section of this course.
 - 45 min — hands-on labs (Part 33), typing every command yourself.
 - 15 min — review yesterday's commands without looking at notes (active recall).
 - 15 min — one troubleshooting scenario (Part 30), talked through out loud as if in an interview.
-

26 Troubleshooting Checklist (Print/Keep Handy)

1. **What changed recently?** (deploys, config edits, scaling events, dependency updates)
 2. **CPU** — `top`, `uptime + nproc` (Part 13.2)
 3. **Memory** — `free -h`, `dmesg | grep -i oom` (Part 12)
 4. **Disk** — `df -h`, `df -i` (Part 11)
 5. **Network** — `ss -tulnp`, `curl -v`, `dig` (Part 14.9)
 6. **Service state** — `systemctl status`, `journalctl -u <svc>` (Part 8.4)
 7. **Permissions/security** — `ls -l`, `id`, SELinux/AppArmor denials (Part 21.3)
 8. **Logs, always** — application logs, `/var/log`, `journalctl`, `dmesg` — in that rough order
 9. **Isolate the layer** — infra (host) vs container vs application code
 10. **Document the root cause and the fix** — every incident should feed back into monitoring/alerting/prevention (the last column of every Part 30 scenario)
-

End of course. This document is meant to be a living reference — re-read Part 30 and re-run Part 33's labs periodically; real fluency comes from repetition against real (or realistically broken) systems, not from a single read-through.